
Technische Schnittstellenbeschreibung

WebSocket-API des Verzeichnisdienstes

Version 0.9

Inhaltsverzeichnis

1.	Einleitung.....	5
1.1	Ziel des Dokuments	5
1.2	Geltungsbereich und Abgrenzung	5
1.3	Schlüsselwörter	6
1.4	Glossar	6
2.	Verbindungsendpunkt und Schnittstellenversion	8
2.1	Endpunkt.....	8
2.2	Umgebungen.....	8
2.3	Versionierung der Schnittstelle	8
2.4	WebSocket-Subprotokoll.....	9
3.	Verbindungsaufbau und Absicherung.....	10
3.1	Transportebene	10
3.2	Authentisierung und Sicherheitskontext.....	10
3.3	HTTP-Header beim Verbindungsaufbau.....	10
3.4	Absicherung des Verbindungsaufbaus	12
3.5	Nachrichtensignatur innerhalb der Verbindung.....	13
3.6	Fehler beim Verbindungsaufbau	14
4.	Nachrichtenmodell	16
4.1	Grundsätze.....	16
4.2	Nachrichtenrahmen (Envelope).....	16
4.3	Nachrichtentypen	18
4.4	Korrelations- und Revisionskennungen.....	19
	4.4.1 Korrelationskennung.....	19
	4.4.2 Revisionskennung.....	20
5.	Subscriptions	21
5.1	Einrichtung einer Subscription (subscribe).....	21
5.2	Bestätigung (subscribeAck)	22
5.3	Kündigung einer Subscription (unsubscribe / unsubscribeAck)	22
5.4	Idempotenz und Verhalten bei mehrfachen Anfragen.....	23
5.5	Mengengerüste.....	23
6.	Benachrichtigungen.....	24

6.1	Nachricht notification	24
6.2	Ereignisarten.....	25
6.3	Umfang der Benachrichtigungen.....	27
6.4	Reihenfolge, Duplikate und Verspätungen	27
7.	Technische Verfügbarkeitsmeldung	28
7.1	Beginn der Verfügbarkeitsmeldung (availabilityStart).....	28
7.2	Fachlicher Heartbeat (availabilityHeartbeat)	29
7.3	Beendigung der Verfügbarkeitsmeldung (availabilityStop).....	29
7.4	Wegfall der Verfügbarkeitsmeldung und Statusbehandlung	31
7.5	Wiederaufnahme der Verfügbarkeit.....	33
7.6	Nachvollziehbarkeit automatischer Statusänderungen	33
7.7	Aktivierung der Verfügbarkeitsüberwachung im Verzeichniseintrag	34
8.	Fehlerbehandlung.....	37
8.1	Fehlernachricht (error)	37
8.2	Fehlercodes	38
8.3	Close-Codes	39
8.4	Protokollfehler auf WebSocket-Ebene	40
9.	Zustell- und Wiederanlaufverhalten	42
9.1	Zustellgarantien.....	42
9.2	Wiederanlauf und Backoff	42
9.3	Resynchronisation über die synchrone Web-API	43
10.	Betriebsparameter.....	43
11.	JSON-Schemata	47
11.1	Validierungsgrundsätze.....	47
11.2	Kennungsformate.....	48
11.3	Client-Nachrichtenschema.....	48
11.4	Server-Nachrichtenschema	49
11.5	Verarbeitungsreihenfolge bei Client-Nachrichten	49
12.	Beispielablauf	51
13.	Quellen	52

Abkürzungsverzeichnis

API	Application Programming Interface / Anwendungsschnittstelle
BDEW	Bundesverband der Energie- und Wasserwirtschaft
BOM	Byte Order Mark
BSI	Bundesamt für Sicherheit in der Informationstechnik
EMT	Externer Marktteilnehmer (Rolle innerhalb der SM-PKI)
H2-	Namespace-Präfix der in der API-Guideline definierten proprietären HTTP-Header
HTTP	Hypertext Transfer Protocol
I-JSON	Internet JSON (restriktives JSON-Profil gemäß RFC 7493)
JSON	JavaScript Object Notation
MP-ID	Marktpartner-ID
mTLS	Mutual Transport Layer Security
PROD	Produktivumgebung
RFC	Request for Comments
SM-PKI	Smart Meter Public Key Infrastructure
SNI	Server Name Indication
TEST	Test- und Abnahmeumgebung
TLS	Transport Layer Security
UTC	Coordinated Universal Time / koordinierte Weltzeit
UTF	Unicode Transformation Format
UUID	Universally Unique Identifier
WSS	WebSocket Secure (WebSocket über TLS)

1. Einleitung

1.1 Ziel des Dokuments

Dieses Dokument ist die technische Schnittstellenbeschreibung der WebSocket-API des zentralen Verzeichnisdienstes am Data Hub im Sinne der API-Guideline Wasserstoffmarkt, Teil 2, Kapitel 5.1 [API-GL].

Es legt gemäß den dortigen Vorgaben verbindlich fest:

- den Verbindungsendpunkt und die Schnittstellenversion,
- die Nachrichtentypen und Nachrichtenschemata,
- Korrelations- und Revisionskennungen,
- die Verwaltung von Subscriptions,
- Fehlernachrichten und Close-Codes,
- das Zustell- und Wiederanlaufverhalten,
- die Parameter für Heartbeat, Timeout und maximale Nachrichtengröße.

Die in der API-Guideline Wasserstoffmarkt getroffenen Regelungen gelten unverändert fort. Dieses Dokument konkretisiert diese Regelungen und wiederholt sie nur, soweit dies für das Verständnis erforderlich ist. Bei Widersprüchen gehen die Regelungen der API-Guideline vor.

1.2 Geltungsbereich und Abgrenzung

Die Beschreibung gilt für die asynchrone WebSocket-API des zentralen Verzeichnisdienstes am Data Hub. Sie richtet sich an alle Marktpartner, die diese Schnittstelle nutzen, sowie an den technischen Betreiber des Verzeichnisdienstes.

Die WebSocket-API unterstützt zwei fachlich getrennte Zwecke:

Subscriptions zum Empfang von Änderungen an Verzeichniseinträgen (Kapitel 5 und 6),

technische Verfügbarkeitsmeldungen für eigene Verzeichniseinträge (Kapitel 7).

Die Nutzung der WebSocket-API durch Clients ist optional. Sie erfolgt ergänzend zur synchronen Abfrage von Verzeichniseinträgen über die Web-API. Eine Benachrichtigung über eine Änderung ersetzt nicht die fachliche Prüfung, ob ein Verzeichniseintrag für einen konkreten Prozess verwendet werden darf.

Die WebSocket-API begründet keine direkte technische Marktkommunikation zwischen Marktpartnern. Sie dient ausschließlich der Kommunikation zwischen berechtigten Clients und dem zentralen Verzeichnisdienst am Data Hub.

1.3 Schlüsselwörter

Die Schlüsselwörter „MUSS“/„MÜSSEN“ (**MUST**), „DARF NICHT“ (**MUST NOT**), „ERFORDERLICH“ (**REQUIRED**), „SOLLTE“ (**SHOULD**), „SOLLTE NICHT“ (**SHOULD NOT**), „EMPFOHLEN“ (**RECOMMENDED**), „NICHT EMPFOHLEN“ (**NOT RECOMMENDED**), „KANN/DARF“ (**MAY**) und „OPTIONAL“ (**OPTIONAL**) sind gemäß [RFC2119] und [RFC8174] zu interpretieren, wenn sie in Großbuchstaben verwendet werden.

1.4 Glossar

Begriff	Beschreibung
Client	Technisches System eines Marktpartners, das eine WebSocket-Verbindung zum Verzeichnisdienst aufbaut.
Server	Der zentrale Verzeichnisdienst am Data Hub in seiner Rolle als Endpunkt der WebSocket-API.
Subscription	Beobachtung eines Verzeichniseintrags mit dem Ziel, über dessen Änderungen benachrichtigt zu werden. Eine Subscription ist keine technische Verfügbarkeitsmeldung.
Technische Verfügbarkeitsmeldung	Meldung des verantwortlichen Systems eines Marktpartners an den Verzeichnisdienst, dass ein bestimmter marktpartnerbezogener API-Webdienst technisch verfügbar ist.
Fachlicher Heartbeat	Zyklisch gesendete Nachricht auf Nachrichtenebene zur Aufrechterhaltung einer technischen Verfügbarkeitsmeldung. Vom technischen WebSocket-Ping/Pong zu unterscheiden.
Revision	Monoton steigende Änderungskennung eines Verzeichniseintrags.
Nachrichtenrahmen (Envelope)	Einheitliche äußere JSON-Struktur, in die alle Nachrichten der WebSocket-API eingebettet sind.
connectionId	Bindungskennung der WebSocket-Verbindung für die Nachrichtensignatur. Sie entspricht der H2-Transaction-Id des erfolgreichen Upgrade-Requests und wird nicht als Envelope-Eigenschaft übertragen.
availabilityMonitoringStatus	Systemseitig geführter Zustand der Aussagekraft der technischen Verfügbarkeitsüberwachung eines Verzeichniseintrags.

Begriff	Beschreibung
Mass-Loss-Circuit-Breaker	Clusterweit wirksamer Schutzmechanismus, der automatische down-Übergänge bei ungewöhnlich vielen korrelierten Wegfällen von Verfügbarkeitsmeldungen unterbindet.

2. Verbindungsendpunkt und Schnittstellenversion

2.1 Endpunkt

Der Client baut über die folgende URI eine TLS-gesicherte WebSocket-Verbindung zum zentralen Verzeichnisdienst auf:

`wss://<Host>/v1/ws/subscriptions`

Dabei ist:

- `<Host>` durch den DNS-Namen des zentralen Verzeichnisdienstes am Data Hub der jeweiligen Umgebung zu ersetzen.

MUST Der Endpunkt MUSS ausschließlich über das Schema `wss` (WebSocket über TLS) bereitgestellt und genutzt werden. Verbindungen über `ws` (ungesichert) DÜRFEN NICHT angeboten oder akzeptiert werden.

MUST Der Verbindungsaufbau MUSS gemäß [RFC6455] als HTTP/1.1-GET-Request mit Upgrade auf WebSocket über TLS erfolgen. Ein Verbindungsaufbau über einen abweichenden HTTP-Mechanismus ist in Version 1 der Schnittstelle nicht vorgesehen.

2.2 Umgebungen

Der Verzeichnisdienst stellt die WebSocket-API in einer Produktivumgebung (PROD) und einer Test- und Abnahmeumgebung (TEST) bereit. Beide Umgebungen sind strikt voneinander getrennt.

MUST PROD und TEST MÜSSEN über unterschiedliche Hostnamen erreichbar sein. Eine wechselseitige Verarbeitung von Nachrichten zwischen den Umgebungen DARF NICHT erfolgen.

MUST Für die in PROD und TEST zu verwendenden Zertifikate gelten die Regelungen und Übergangsbestimmungen gemäß Teil 2, Kapitel 2.4 der API-Guideline [API-GL]. Abweichende oder verkürzende Festlegungen trifft diese Schnittstellenbeschreibung nicht.

2.3 Versionierung der Schnittstelle

Die Hauptversion der WebSocket-API ist Bestandteil des URI-Pfads (v1). Für die Versionierung gelten die Regelungen aus Teil 1, Kapitel 4.2 der API-Guideline [API-GL] entsprechend.

MUST Abwärtskompatible Änderungen (z. B. neue optionale Eigenschaften, neue Server-Nachrichtentypen, neue Ereignisarten oder neue Fehlercodes) MÜSSEN innerhalb derselben Hauptversion erfolgen. Clients MÜSSEN unbekannte optionale Eigenschaften und unbekannte

Nachrichtentypen des Servers tolerieren. Unbekannte Ereignisarten innerhalb einer *notification* DÜRFEN NICHT als Protokollfehler behandelt werden; ist die übermittelte *revision* höher als der lokale Stand, MUSS der Client den Eintrag über die synchrone Web-API resynchronisieren (Tolerant Reader; Kapitel 6.4 und 11.1). Nachrichten des Clients an den Server werden strikt gegen die geschlossenen Client-Schemata validiert.

MUST Nicht abwärtskompatible Änderungen MÜSSEN zu einer neuen Hauptversion mit eigenem URI-Pfad (z. B. /v2/ws/subscriptions) führen. Der Parallelbetrieb und die Übergangsfristen richten sich nach dem Änderungsmanagement gemäß Teil 1, Kapitel 4.2 [API-GL].

2.4 WebSocket-Subprotokoll

MUST Der Client MUSS beim Verbindungsaufbau im Header `Sec-WebSocket-Protocol` das Subprotokoll `h2.directory.v1` anfordern. Der Server MUSS dieses Subprotokoll bestätigen. Kommt keine Einigung auf das Subprotokoll zustande, MUSS der Server den Verbindungsaufbau ablehnen.

`Sec-WebSocket-Protocol: h2.directory.v1`

3. Verbindungsaufbau und Absicherung

3.1 Transportebene

MUST Die WebSocket-Verbindung MUSS ausschließlich über WSS und als mTLS-Verbindung gemäß Teil 2, Kapitel 2.2 der API-Guideline [API-GL] aufgebaut werden. Die kryptographischen Vorgaben der [TR03116-3] MÜSSEN eingehalten werden.

MUST Der Client MUSS beim Verbindungsaufbau das gültige TLS-Client-Zertifikat seines EMT-API-Zertifikatstripels verwenden. Das Feld Organizational Unit des Subject MUSS die MP-ID des Marktpartners enthalten.

MUST Für den Aufbau der TLS-Verbindung MUSS die Erweiterung Server Name Indication (SNI) gemäß [RFC6066] unterstützt und verwendet werden.

MUST Server und Client MÜSSEN die Gültigkeit des jeweils präsentierten Zertifikats einschließlich der Sperrlisten (CRL) gemäß der Certificate Policy der SM-PKI [CP-SM-PKI] prüfen.

MUST NOT Verbindungen mit einem ungültigen, abgelaufenen oder gesperrten TLS- oder gebundenen SIG-Zertifikat DÜRFEN NICHT aufgebaut oder aufrechterhalten werden. Wird die Sperrung oder der Ablauf eines dieser Zertifikate während einer bestehenden Verbindung festgestellt, MUSS der Server die Verbindung mit Close-Code 4001 beenden.

3.2 Authentisierung und Sicherheitskontext

MUST Der Server MUSS den Marktpartner aus der mTLS-Authentisierung eindeutig anhand der MP-ID im Client-Zertifikat bestimmen.

MUST Der durch die mTLS-Authentisierung hergestellte Sicherheitskontext MUSS für die gesamte Dauer der jeweiligen WebSocket-Verbindung gelten. Der Server MUSS zusätzlich die im Upgrade-Request angegebene H2-Message-Sender-Rolle gegen das Rollen- und Rechtemodell prüfen und zusammen mit der MP-ID an die Verbindung binden. Sämtliche über die Verbindung verwalteten Subscriptions und technischen Verfügbarkeitsmeldungen MÜSSEN ausschließlich entsprechend diesem Rollen- und Sicherheitskontext verarbeitet werden.

MUST NOT Eine Änderung des Rollen- oder Sicherheitskontexts innerhalb einer bestehenden Verbindung DARF NICHT erfolgen. Ein Wechsel des TLS- oder SIG-Zertifikats oder der Marktrolle erfordert den Aufbau einer neuen Verbindung. Marktpartner, die mehrere Marktrolle nutzen, MÜSSEN dafür getrennte Verbindungen mit jeweils eindeutigem Rollenbezug aufbauen.

3.3 HTTP-Header beim Verbindungsaufbau

Der HTTP-Request zum Aufbau der WebSocket-Verbindung (Upgrade-Request) MUSS die folgenden Header enthalten. Für die Namenskonvention gilt Teil 1, Kapitel 4.5 der API-Guideline [API-GL].

HTTP-Header	Pflicht	Beschreibung
Host	MUSS	DNS-Name des Verzeichnisdienstes der jeweiligen Umgebung.
Upgrade	MUSS	Fester Wert websocket gemäß [RFC6455].
Connection	MUSS	Fester Wert Upgrade gemäß [RFC6455].
Sec-WebSocket-Key	MUSS	Zufallswert gemäß [RFC6455].
Sec-WebSocket-Version	MUSS	Fester Wert 13 gemäß [RFC6455].
Sec-WebSocket-Protocol	MUSS	Angefordertes Subprotokoll h2.directory.v1 (siehe Kapitel 2.4).
H2-Transaction-Id	MUSS	UUID Version 7 gemäß Teil 1, Kapitel 4.5.2 [API-GL] zur eindeutigen Identifikation des Verbindungsaufbaus. Der Wert dient zugleich als connectionId des Signaturkontexts der aufgebauten Verbindung.
H2-Cert	MUSS	DER-kodiertes SIG-Zertifikat des EMT.API-Zertifikatstripels als Byte Sequence Structured Field gemäß [RFC9440]. Es wird geprüft und für die Dauer der Verbindung an den Sicherheitskontext gebunden. Maximale Länge: 16.384 Byte gemäß [JWS-SPEC], Kapitel 5.3.
H2-Message-Sender	MUSS	MP-ID des aufrufenden Marktpartners (Client). MUSS mit der MP-ID im Client-Zertifikat übereinstimmen.
H2-Message-Sender-Role	MUSS	Marktrolle des aufrufenden Marktpartners gemäß Teil 1, Kapitel 4.5.2 [API-GL].
H2-Message-Receiver	MUSS	MP-ID des Data Hub als Betreiber des Verzeichnisdienstes.
H2-Message-Receiver-Role	MUSS	Fester Wert DataHub.
H2-Business-Process	MUSS	Fester Wert directoryWebSocket. Der Wert MUSS durch die API-Governance als zulässiger H2-Business-Process registriert sein.

MUST Der Upgrade-Request MUSS sämtliche gemäß Teil 1, Kapitel 4.5.2 [API-GL] verbindlichen Request-Header enthalten. Für H2-Message-Receiver-Role und H2-Business-Process gelten die in der Tabelle genannten festen Werte.

MUST Jede nach erfolgreicher TLS-Authentisierung erzeugte HTTP-Antwort MUSS den Header H2-API-Version enthalten. Für diese Fassung lautet der Wert 1.0.0. Die erfolgreiche Antwort mit HTTP-Statuscode 101 Switching Protocols MUSS zusätzlich Upgrade: websocket, Connection: Upgrade und Sec-WebSocket-Accept gemäß [RFC6455] enthalten sowie das vereinbarte Subprotokoll mit Sec-WebSocket-Protocol: h2.directory.v1 bestätigen.

MUST Jeder Verbindungsaufbau einschließlich einer Wiederverbindung nach einer beendeten Verbindung MUSS eine neue H2-Transaction-Id verwenden. Ein Wiederverbindungsversuch ist ein neuer Verbindungsaufbau und kein Retry-Request im Sinne von Teil 1, Kapitel 4.5.3 [API-GL].

MUST NOT H2-Initial-Transaction-Id DARF beim Upgrade-Request NICHT übertragen werden (Teil 1, Kapitel 4.5.3 [API-GL]).

MUST Client und Server MÜSSEN die H2-Transaction-Id des erfolgreich abgeschlossenen Upgrade-Requests für die Dauer der Verbindung als `connectionId` des Signaturkontexts binden.

MUST Der Server MUSS einen Verbindungsaufbau mit `409 Conflict` ablehnen, wenn dieselbe H2-Transaction-Id für dieselbe MP-ID und Umgebung bereits an eine aktive Verbindung gebunden ist oder innerhalb der letzten 600 Sekunden als `connectionId` verwendet wurde.

MUST Die Dauer einer WebSocket-Verbindung DARF 24 Stunden NICHT überschreiten. Der Server MUSS die Verbindung spätestens nach Ablauf von 24 Stunden mit Close-Code `4005` planmäßig beenden.

SHOULD Der Server SOLLTE die planmäßigen Rotationszeitpunkte der aktiven Verbindungen einer Umgebung zeitlich streuen, sodass nicht mehr als 10 Prozent der aktiven Verbindungen innerhalb desselben 60-Sekunden-Fensters mit Close-Code `4005` beendet werden.

MUST Nach Close-Code `4005` MUSS der Client eine neue Verbindung mit einer neuen H2-Transaction-Id und damit einer neuen `connectionId` aufbauen. Vor dem ersten Wiederverbindungsversuch MUSS er einen zufälligen, gleichverteilten Jitter von 1 bis 30 Sekunden anwenden.

3.4 Absicherung des Verbindungsaufbaus

Der HTTP-Request zum Aufbau der WebSocket-Verbindung wird nicht auf Inhaltsdatenebene signiert. Seine Authentisierung erfolgt durch mTLS. Der Header `H2-Cert` überträgt ausschließlich das für spätere WebSocket-Nachrichten vorgesehene SIG-Zertifikat und ist nicht selbst eine Nachrichtensignatur.

MUST NOT Auch die erfolgreiche Response `101 Switching Protocols` und negative HTTP-Responses des Upgrade-Verfahrens DÜRFEN NICHT auf Inhaltsdatenebene signiert werden. Diese Ausnahme gilt ausschließlich für den Aufbau der WebSocket-Verbindung; synchrone Responses fachlicher HTTP-API-Aufrufe mit JSON-Body richten sich nach [JWS-SPEC].

MUST NOT Der Upgrade-Request DARF KEINEN HTTP-Body enthalten.

MUST Der Server MUSS vor Abschluss des Upgrades die Prüfungen gemäß Kapitel 3.1 bis 3.3 durchführen. Zusätzlich MUSS er H2-Cert syntaktisch dekodieren, das enthaltene SIG-Zertifikat einschließlich Kette, Gültigkeit, Sperrstatus und Signaturverwendungszweck prüfen und feststellen, dass SIG- und TLS-Zertifikat demselben EMT-API-Zertifikatstripel und derselben MP-ID zugeordnet sind.

3.5 Nachrichtensignatur innerhalb der Verbindung

Gemäß Teil 2, Kapitel 5.2 der API-Guideline [API-GL] gilt für Nachrichten innerhalb der WebSocket-Verbindung:

MUST Client-Nachrichten, die Subscriptions einrichten oder kündigen oder als technische Verfügbarkeitsmeldung dienen, MÜSSEN mit dem privaten Schlüssel des beim Upgrade gebundenen SIG-Zertifikats auf Nachrichtenebene signiert werden. Die Signatur wird in der Envelope-Eigenschaft `signature` übertragen. Signaturbildung, Kanonisierung, Zeitfenster und Prüfverfahren richten sich nach [JWS-SPEC].

MUST Die Signaturrepräsentation MUSS gemäß [JWS-SPEC] die `connectionId` der aktuellen Verbindung und den Envelope ohne `signature` enthalten.

MUST `connectionId` DARF NICHT als zusätzliche Eigenschaft im Envelope übertragen werden. Der Client MUSS den Wert aus seinem erfolgreichen Upgrade-Request und der Server aus dem gebundenen Verbindungskontext verwenden.

MUST Eine Signatur, die für eine andere `connectionId` erzeugt wurde, MUSS mit `ERR-SIG-001` abgelehnt werden.

MUST Für das Signaturzeitfenster einer WebSocket-Nachricht ist ausschließlich der signierte `timestamp` des Envelopes maßgeblich. Eine Eigenschaft `created` wird im WebSocket-Protokoll NICHT verwendet.

MUST Der Server MUSS die Signatur jeder signaturpflichtigen Nachricht vor der Zustandsänderung prüfen. Nachrichten mit fehlender, strukturell ungültiger oder kryptographisch ungültiger Signatur MUSS er mit `ERR-SIG-001` ablehnen; der Zustand von Subscriptions und Verfügbarkeitsmeldungen DARF hierdurch NICHT verändert werden.

SHOULD Die Prüfung der Zertifikatskette und des Sperrstatus des gebundenen SIG-Zertifikats SOLLTE der Server je Verbindung und `kid` zwischenspeichern. Die Zwischenspeicherung DARF höchstens bis zur Aktualitätsgrenze der verwendeten Sperrliste, bis zum Ablauf des Zertifikats oder bis zu einer Änderung des Truststores gültig sein.

MUST Der `kid` jeder signaturpflichtigen Client-Nachricht MUSS dem Thumbprint des beim Upgrade gebundenen SIG-Zertifikats entsprechen. Ein Wechsel dieses Zertifikats erfordert einen neuen Verbindungsaufbau.

MUST NOT Nachrichten des Servers an den Client (`subscribeAck`, `unsubscribeAck`, `availabilityAck`, `notification`, `error`) werden NICHT auf Nachrichtenebene signiert; die Envelope-Eigenschaft `signature` DARF in ihnen NICHT enthalten sein. Ihre Absicherung erfolgt durch die mTLS-gesicherte Transportverbindung.

MUST Benachrichtigungen der WebSocket-API haben ausschließlich informatorischen Charakter. Soweit ein Client den Inhalt eines Verzeichniseintrags fachlich verwendet, MUSS er den Eintrag über die synchrone Web-API des Verzeichnisdienstes beziehen beziehungsweise verifizieren; dort gelten die Regelungen zur Inhaltsdatensicherung unverändert.

3.6 Fehler beim Verbindungsaufbau

Schlägt der Verbindungsaufbau fehl, antwortet der Server mit einem HTTP-Statuscode gemäß nachfolgender Tabelle. Ein WebSocket-Upgrade findet in diesen Fällen nicht statt.

HTTP-Statuscode	Bedeutung
400 Bad Request	Fehlerhafter Upgrade-Request, fehlende oder ungültige Pflicht-Header, syntaktisch ungültiger H2-Cert oder kein unterstütztes WebSocket-Subprotokoll.
401 Unauthorized	Anwendungsauthentisierung fehlgeschlagen, z. B. ungültiges, gesperrtes oder ungeeignetes SIG-Zertifikat oder Abweichung der MP-ID zwischen H2-Message-Sender, TLS- und SIG-Zertifikat. Scheitert bereits mTLS, wird keine HTTP-Antwort erzeugt.
403 Forbidden	Der authentifizierte Marktpartner oder die angegebene Marktrolle ist für die Nutzung der WebSocket-API nicht berechtigt.
404 Not Found	Unbekannter Pfad oder nicht vorhandene Hauptversion.
405 Method Not Allowed	Andere HTTP-Methode als GET. Der Header Allow MUSS den Wert GET enthalten.
409 Conflict	Die H2-Transaction-Id des Upgrade-Requests ist bereits als aktive oder noch gesperrte connectionId verwendet worden.
410 Gone	Die adressierte Hauptversion wurde dauerhaft außer Betrieb genommen.
426 Upgrade Required	Die angeforderte WebSocket-Version wird nicht unterstützt. Der Server MUSS Sec-WebSocket-Version: 13 zurückgeben.
429 Too Many Requests	Überschreitung der zulässigen Anzahl gleichzeitiger Verbindungen oder der Rate von Verbindungsaufbauversuchen (Kapitel 10).
500 Internal Server Error	Unerwarteter interner Fehler vor Abschluss des Upgrades.
503 Service Unavailable	Der Dienst steht vorübergehend nicht zur Verfügung, z. B. wegen Wartung oder Überlastung.

MUST Negative HTTP-Antworten mit Response-Body MÜSSEN dem ErrorResponse-Schema gemäß Teil 1, Kapitel 4.4.3 [API-GL] entsprechen; transactionId MUSS die H2-Transaction-

Id des abgelehnten Upgrade-Requests enthalten. Kann aus dem Upgrade-Request keine syntaktisch gültige H2-Transaction-Id bestimmt werden, MUSS die Ablehnung ohne ErrorResponse-Body und ohne H2-Reference-Id erfolgen.

MUST Eine negative HTTP-Antwort des Upgrade-Verfahrens mit ErrorResponse-JSON-Body MUSS eine neu erzeugte H2-Transaction-Id der Response sowie H2-Reference-Id, H2-API-Version und Content-Type: application/json enthalten. Sie DARF die Header H2-Signature, H2-Digest und H2-Cert NICHT enthalten. transactionId im ErrorResponse-Body MUSS mit H2-Reference-Id übereinstimmen.

SHOULD Bei den HTTP-Statuscodes 429 und 503 SOLLTE der Server Retry-After setzen. Bei 426 MUSS der Server Sec-WebSocket-Version: 13 setzen. Ein Client MUSS einen gültigen Retry-After-Wert beim Wiederanlauf gemäß Kapitel 9.2 berücksichtigen.

4. Nachrichtenmodell

4.1 Grundsätze

MUST Alle Nachrichten MÜSSEN als WebSocket-Textframes mit UTF-8-kodiertem JSON ohne BOM übertragen werden. Binärframes DÜRFEN NICHT verwendet werden.

MUST Für alle Nachrichten gelten die Anforderungen aus Teil 1, Kapitel 4.3 der API-Guideline [API-GL] – insbesondere I-JSON-Konformität gemäß [RFC7493], die dort definierten Datentypen und Formate sowie die Validierbarkeit gegen das jeweils gültige Nachrichtenschema.

MUST NOT Eine Client- oder Server-Nachricht DARF auf keiner Objektebene mehrfach vorkommende JSON-Eigenschaftsnamen enthalten. Empfänger MÜSSEN Duplikate vor Schema- und Signaturverarbeitung erkennen; Client-Nachrichten mit Duplikaten sind mit `ERR-VAL-001` abzulehnen.

MUST NOT Eine Nachricht einschließlich des Nachrichtenrahmens DARF die maximale Nachrichtengröße von 1.048.576 Byte (1 MiB) NICHT überschreiten. Empfängt der Server eine größere Nachricht, MUSS er die Verbindung mit Close-Code 1009 beenden. Der Server DARF selbst KEINE Nachricht oberhalb dieser Grenze senden.

MUST NOT Der als UTF-8 serialisierte Wert der Eigenschaft `entry` DARF 921.600 Byte (900 KiB) NICHT überschreiten.

MUST Der Verzeichnisdienst MUSS diese Grenze bereits bei der Anlage oder Änderung eines Verzeichniseintrags durchsetzen, damit vollständige Benachrichtigungen innerhalb der maximalen Nachrichtengröße übertragen werden können.

MUST NOT Empfänger DÜRFEN die Reihenfolge von JSON-Feldern NICHT fachlich auswerten. Die Feldreihenfolge ist nicht Bestandteil des Vertrags (Teil 1, Kapitel 4.4.2 [API-GL]).

4.2 Nachrichtenrahmen (Envelope)

Jede Nachricht der WebSocket-API ist in einen einheitlichen Nachrichtenrahmen eingebettet:

```
{
  "type": "subscribe",
  "messageId": "3f2a6c1e-9d4b-4c1a-8e2f-5b7d9a0c3e11",
  "correlationId": null,
  "timestamp": "2026-07-12T09:30:00Z",
  "signature":
    "eyJhbGciOiJI1cm46aDI6andzOmFsZzpicmFpbmBvb2xQMjU2cjEtc2hhMjU2IiwidHlwIjoiaDItd3Mtc2lnK2p3cyIsImI2NCI6ZmFsc2UsImNyaXQiOlsiYjY0I0sImtpZCI6Im5TbFkwblo4LWV4YW1wbGUifQ..gekuerzt",
  "payload": { "providerId": "9910123400002", "apiId": "H2API00001", "majorVersion":
1 }
```

}

Eigenschaft	Typ / Format	Pflicht	Beschreibung
type	string	MUSS	Nachrichtentyp gemäß Kapitel 4.3. Für Server-Nachrichten können innerhalb derselben Hauptversion weitere Typen hinzukommen.
messageId	string (uuid)	MUSS	Eindeutige Kennung der Nachricht als UUID Version 4 gemäß [RFC9562]. Jede Übertragung erhält eine neue messageId; sie ist keine H2-Transaction-Id. Zusammen mit der verbindungsgebundenen connectionId schützt sie vor Wiederverwendung.
correlationId	string (uuid) null	MUSS	Korrelationskennung gemäß Kapitel 4.4. Bei Nachrichten, die sich nicht auf eine vorherige Nachricht beziehen, ist der Wert null.
timestamp	string (date-time)	MUSS	Erstellungszeitpunkt der Nachricht gemäß [RFC3339] in UTC mit dem Suffix Z. Der Wert ist nicht für Reihenfolge, fachliche Timeouts oder die Verfügbarkeitsfeststellung maßgeblich; ausgenommen ist das Signaturzeitfenster gemäß Kapitel 3.5 und [JWS-SPEC], Kapitel 8.1.
payload	object	MUSS	Typspezifischer Nachrichteninhalt gemäß Kapitel 5 bis 8. Ein leerer Inhalt ist als leeres Objekt {} zu übertragen.
signature	string	BEDINGT	Detached JWS über die kanonisierte Nachricht gemäß [JWS-SPEC]. MUSS bei den signaturpflichtigen Client-Nachrichten gemäß Kapitel 3.5 enthalten sein; DARF bei Server-Nachrichten NICHT enthalten sein. Die Eigenschaft selbst ist nicht Bestandteil der signierten Repräsentation.

MUST Nachrichten des Clients an den Server MÜSSEN auf Rahmen- und Payload-Ebene als geschlossene JSON-Objekte modelliert sein; zusätzliche Eigenschaften sind unzulässig. Die Eigenschaft `signature` gehört zur zulässigen Rahmenstruktur der signaturpflichtigen Client-Nachrichten (Kapitel 3.5). Nachrichten des Servers an den Client beschreiben dagegen eine verbindliche Mindeststruktur und DÜRFEN zusätzliche optionale Eigenschaften enthalten, die Clients gemäß Kapitel 11.1 ignorieren MÜSSEN.

MUST Jede `messageId` MUSS innerhalb einer WebSocket-Verbindung eindeutig sein.

MUST Der Server DARF die `messageId` vor der erfolgreichen Signaturprüfung lediglich unverbindlich auf eine mögliche Wiederverwendung prüfen. Eine dauerhafte Reservierung DARF zu diesem Zeitpunkt NICHT erfolgen.

MUST Nach erfolgreicher Signatur- und Zeitfensterprüfung MUSS der Server die `messageId` atomar für die aktuelle `connectionId` reservieren. Ist sie bereits reserviert, MUSS er `ERR-MSG-002` senden und die Nachricht NICHT erneut verarbeiten.

MUST NOT Eine Nachricht mit fehlender, strukturell ungültiger oder kryptographisch ungültiger Signatur DARF eine `messageId` NICHT dauerhaft reservieren.

MUST Der Server MUSS jede nach erfolgreicher Signatur- und Zeitfensterprüfung reservierte `messageId` für die vollständige Dauer der aktuellen WebSocket-Verbindung vorhalten. Eine Entfernung aus dem Prüfgedächtnis vor dem Ende der Verbindung ist unzulässig.

MUST Die serverseitige Dimensionierung MUSS die Vorhaltung von bis zu 432.000 `messageIds` je Verbindung berücksichtigen (maximale Nachrichtenrate von 50 Nachrichten je 10 Sekunden über die maximale Verbindungsdauer von 24 Stunden, Kapitel 10).

MAY Nach dem Ende der Verbindung DARF der Server den zugehörigen `messageId`-Prüfzustand entfernen. Die `connectionId`-Nachnutzungssperre gemäß Kapitel 3.3 und Kapitel 10 bleibt hiervon unberührt.

MUST Für die fachliche Reihenfolge und alle Timeout-Berechnungen sind die `revision` beziehungsweise der Empfangszeitpunkt beim Server maßgeblich.

MUST NOT Der `timestamp` einer Client-Nachricht DARF insbesondere NICHT zur Bestimmung des Heartbeat-Timeouts verwendet werden.

4.3 Nachrichtentypen

Die WebSocket-API definiert die folgenden Nachrichtentypen:

Nachrichtentyp	Richtung	Beschreibung	Kapitel
<code>subscribe</code>	Client → Server	Einrichtung einer Subscription auf einen Verzeichniseintrag.	5.1
<code>subscribeAck</code>	Server → Client	Bestätigung einer Subscription einschließlich aktueller Revision.	5.2
<code>unsubscribe</code>	Client → Server	Kündigung einer Subscription.	5.3
<code>unsubscribeAck</code>	Server → Client	Bestätigung der Kündigung.	5.3
<code>notification</code>	Server → Client	Benachrichtigung über ein Ereignis zu einem abonnierten Verzeichniseintrag.	6
<code>availabilityStart</code>	Client → Server	Beginn einer technischen Verfügbarkeitsmeldung.	7.1

Nachrichtentyp	Richtung	Beschreibung	Kapitel
availabilityHeartbeat	Client → Server	Fachlicher Heartbeat zur Aufrechterhaltung einer Verfügbarkeitsmeldung.	7.2
availabilityStop	Client → Server	Ordnungsgemäße Beendigung einer Verfügbarkeitsmeldung.	7.3
availabilityAck	Server → Client	Bestätigung von availabilityStart, availabilityHeartbeat und availabilityStop.	7
error	Server → Client	Strukturierte Fehlernachricht.	8.1

MUST Der Server MUSS einen vom Client gesendeten unbekannten oder in Client-Richtung unzulässigen Nachrichtentyp mit einer error-Nachricht (ERR-MSG-001) beantworten. Bestehende Subscriptions und Verfügbarkeitsmeldungen DÜRFEN hierdurch NICHT verändert werden.

MUST Clients MÜSSEN unbekannte, vom Server gesendete Nachrichtentypen ohne Protokollfehler ignorieren. Der Zustand bereits bekannter Subscriptions und Verfügbarkeitsmeldungen DARF dadurch NICHT verändert werden.

4.4 Korrelations- und Revisionskennungen

4.4.1 Korrelationskennung

MUST Antwortet der Server auf eine gültig identifizierbare Nachricht des Clients (subscribeAck, unsubscribeAck, availabilityAck oder error), MUSS er in correlationId die messageId der auslösenden Client-Nachricht übertragen. Kann aus einer fehlerhaften Nachricht keine gültige messageId ermittelt werden, MUSS eine error-Nachricht correlationId = null enthalten.

MUST Ereignisbasierte Benachrichtigungen (notification), die nicht unmittelbar auf eine Client-Nachricht antworten, MÜSSEN correlationId mit dem Wert null übertragen.

SHOULD Clients SOLLTEN offene Anfragen anhand der correlationId eingehenden Antworten zuordnen und nach Ablauf des Antwort-Timeouts (Kapitel 10) als unbeantwortet behandeln.

MUST Der Server MUSS jede gültig empfangene Client-Nachricht innerhalb des Antwort-Timeouts von 30 Sekunden (Kapitel 10) mit der zugehörigen Bestätigungs- oder error-Nachricht beantworten.

MUST Läuft der Antwort-Timeout ab, DARF der Client dieselbe logische Operation mit einer neuen messageId wiederholen. Sämtliche Client-Operationen dieser Schnittstelle MÜSSEN serverseitig idempotent verarbeitet werden.

4.4.2 Revisionskennung

MUST Jeder Verzeichniseintrag MUSS eine Revisionskennung `revision` als Ganzzahl im Bereich 1 bis 9.007.199.254.740.991 führen. Bei der erfolgreichen initialen Anlage MUSS `revision` den Wert 1 erhalten. Jede danach atomar abgeschlossene Änderung des Verzeichniseintrags MUSS die Revision genau einmal und streng monoton erhöhen; mehrere im selben Änderungsvorgang geänderte Eigenschaften erzeugen nur eine neue Revision.

MUST Der Server MUSS die aktuelle Revision in jeder `subscribeAck`- und jeder `notification`-Nachricht übertragen.

MUST Eine `notification` mit `eventType = subscriptionEnded` ändert den Verzeichniseintrag nicht und erhöht dessen Revision nicht. Sie MUSS die zum Zeitpunkt der Beendigung aktuelle Revision enthalten.

MUST Wird ein Verzeichniseintrag gelöscht, MUSS der Verzeichnisdienst die zuletzt vergebene Revision für den Schlüssel `providerId/apiId/majorVersion` so lange aufbewahren, wie eine spätere Wiederanlage unter demselben Schlüssel zulässig ist.

SHOULD Clients SOLLTEN empfangene Revisionen mit dem lokalen Stand vergleichen. Benachrichtigungen mit einer Revision kleiner oder gleich dem lokalen Stand SOLLTEN als Duplikat oder verspätete Nachricht behandelt und ohne Zustandsänderung verworfen werden. Bei Lücken in der Revisionsfolge SOLLTE der Client den Eintrag über die synchrone Web-API resynchronisieren (Kapitel 9.3).

5. Subscriptions

Die WebSocket-API basiert auf dem Konzept sogenannter Subscriptions. Eine Subscription bezieht sich eindeutig auf einen Verzeichniseintrag, der durch die Parameter `providerId` (MP-ID), `apiId` (API-Kennung) und `majorVersion` (Hauptversion) identifiziert wird. Eine Subscription ist eine Beobachtung eines Verzeichniseintrags; sie ist keine technische Verfügbarkeitsmeldung (Teil 2, Kapitel 5.3.1 [API-GL]).

MUST Jeder authentifizierte Marktpartner IST berechtigt, eine Subscription auf jeden im Verzeichnisdienst geführten Verzeichniseintrag einzurichten, unabhängig vom verantwortlichen Marktpartner und unabhängig von den im Verzeichniseintrag angegebenen zulässigen Markttrollen. Die Sichtbarkeit von Verzeichnismetadaten über eine Subscription ist von der Berechtigung zur Nutzung des durch den Verzeichniseintrag beschriebenen API-Webdienstes zu unterscheiden; Letztere richtet sich unverändert nach den zulässigen Markttrollen des jeweiligen Eintrags.

5.1 Einrichtung einer Subscription (subscribe)

Aufbau der Nachricht `subscribe`:

```
{
  "type": "subscribe",
  "messageId": "3f2a6c1e-9d4b-4c1a-8e2f-5b7d9a0c3e11",
  "correlationId": null,
  "timestamp": "2026-07-12T09:30:00Z",
  "signature":
    "eyJhbGciOiJI1cm46aDI6andzOmFsZzpicmFpbmBvb2xQMjU2cjEtc2hhMjU2IiwidHlwIjoiaDItd3Mtc2lnK2p3cyIsImI2NCI6ZmFsc2UsImNyaXQiOlsiYjY0I10sImtpZCI6Im5TbFkwblo4LWV4YW1wbGUifQ..gekuerzt",
  "payload": {
    "providerId": "9910123400002",
    "apiId": "H2API00001",
    "majorVersion": 1
  }
}
```

Eigenschaft	Typ / Format	Pflicht	Beschreibung
<code>providerId</code>	string	MUSS	MP-ID des Marktpartners, der den Verzeichniseintrag verantwortet. Format: 13-stellige numerische MP-ID.
<code>apiId</code>	string	MUSS	API-Kennung gemäß Teil 2, Kapitel 4.2 [API-GL]. Format: H2API gefolgt von fünf Ziffern (z. B. H2API00001).
<code>majorVersion</code>	integer (int32)	MUSS	Hauptversion des API-Webdienstes (positive Ganzzahl).

MUST Der Server MUSS eingehende subscribe-Nachrichten verarbeiten und dem Client das Ergebnis mit subscribeAck oder error mitteilen. Eine Subscription gilt erst mit Empfang der subscribeAck als aktiv.

MUST Der Server MUSS unzulässige Subscription-Anfragen mit einer error-Nachricht ablehnen. Existiert der referenzierte Verzeichniseintrag nicht, MUSS der Server ERR-SUB-001 verwenden; da gemäß Kapitel 5 jeder authentifizierte Marktpartner zum Empfang jedes Verzeichniseintrags berechtigt ist, ist mit der Verwendung von ERR-SUB-001 keine unberechtigte Existenzoffenlegung verbunden. Der Fehlercode ERR-AUTH-001 findet auf subscribe-Anfragen keine Anwendung.

5.2 Bestätigung (subscribeAck)

```
{
  "type": "subscribeAck",
  "messageId": "b1e7c9d2-4a6f-4e3b-9c0d-2f8a1b5e6d47",
  "correlationId": "3f2a6c1e-9d4b-4c1a-8e2f-5b7d9a0c3e11",
  "timestamp": "2026-07-12T09:30:01Z",
  "payload": {
    "providerId": "9910123400002",
    "apiId": "H2API00001",
    "majorVersion": 1,
    "revision": 17
  }
}
```

MUST Die subscribeAck MUSS die Revision des Verzeichniseintrags enthalten, die der Server beim Aktivieren der Subscription als konsistenten Ausgangsstand festgestellt hat.

MUST Die Aktivierung der Subscription und die Feststellung der in subscribeAck übertragenen Ausgangsrevision MÜSSEN einen atomaren Snapshot-/Event-Barrier-Mechanismus verwenden. Keine nach dieser Ausgangsrevision verbindlich abgeschlossene Änderung DARF zwischen Aktivierung und Beginn der Notification-Zustellung verloren gehen.

MUST Der Server MUSS die subscribeAck vor jeder notification für die neu aktivierte Subscription senden. Änderungen mit einer höheren Revision als der in subscribeAck übermittelte Ausgangsstand MÜSSEN anschließend in aufsteigender Revisionsfolge benachrichtigt werden.

SHOULD Der Client SOLLTE die übermittelte Revision mit seinem lokalen Stand vergleichen und bei Abweichung den Verzeichniseintrag über die synchrone Web-API abrufen (Kapitel 9.3).

5.3 Kündigung einer Subscription (unsubscribe / unsubscribeAck)

Die Kündigung erfolgt durch die Nachricht unsubscribe mit demselben Payload-Schema wie subscribe (providerId, apiId, majorVersion). Der Server bestätigt mit unsubscribeAck.

MUST Eine Subscription gilt für die Dauer der WebSocket-Verbindung oder bis zu ihrer Kündigung. Der Client KANN eine Subscription jederzeit kündigen.

MUST Mit dem Ende der WebSocket-Verbindung enden sämtliche über diese Verbindung eingerichteten Subscriptions. Eine serverseitige Persistierung von Subscriptions über das Verbindungsende hinaus erfolgt NICHT.

MUST Nach dem Versand einer unsubscribeAck DARF der Server keine weitere notification für die gekündigte Subscription senden. Bereits zur Übertragung vorgesehene Benachrichtigungen MÜSSEN entweder vor der unsubscribeAck gesendet oder verworfen werden. Bei einer Kündigung durch den Client wird KEINE zusätzliche notification mit eventType = subscriptionEnded gesendet.

5.4 Idempotenz und Verhalten bei mehrfachen Anfragen

MUST Die Verwaltung von Subscriptions MUSS idempotent erfolgen. Mehrfache subscribe-Nachrichten für denselben Verzeichniseintrag innerhalb derselben Verbindung DÜRFEN NICHT zu mehrfachen Subscriptions führen; der Server MUSS jede dieser Nachrichten gleichwohl mit subscribeAck (einschließlich aktueller Revision) beantworten.

MUST NOT Die Kündigung einer nicht existierenden Subscription DARF NICHT als Fehler behandelt werden. Der Server MUSS in diesem Fall mit unsubscribeAck antworten.

5.5 Mengengerüste

MUST Je WebSocket-Verbindung DÜRFEN höchstens 1.000 gleichzeitig aktive Subscriptions bestehen. Darüber hinausgehende subscribe-Nachrichten MUSS der Server mit dem Fehlercode ERR-SUB-002 ablehnen. Eine idempotente Wiederholung für eine bereits aktive Subscription legt keine zusätzliche Subscription an und DARF wegen des ausgeschöpften Limits NICHT abgelehnt werden.

SHOULD Clients SOLLTEN unnötige Subscriptions vermeiden und nicht mehr benötigte Subscriptions zeitnah kündigen.

6. Benachrichtigungen

6.1 Nachricht notification

MUST Der Server MUSS Clients über Änderungen an Verzeichniseinträgen informieren, sofern eine aktive Subscription besteht. Die Benachrichtigung MUSS im regulären Betrieb spätestens 30 Sekunden nach dem verbindlichen Abschluss der Änderung an die WebSocket-Verbindung übergeben werden (Kapitel 10).

MUST Eine Änderungs-notification DARF ausschließlich aus einem nach erfolgreichem Commit erzeugten Transactional-Outbox-Ereignis gebildet werden. Ein nicht erfolgreich abgeschlossener oder zurückgerollter Änderungsvorgang DARF keine Notification erzeugen.

MUST Jedes Outbox-Ereignis MUSS durch Verzeichnisschlüssel und `revision` eindeutig identifizierbar sein. Die Weitergabe an eine konkrete WebSocket-Verbindung MUSS idempotent erfolgen, sodass ein wiederholter Verarbeitungsversuch innerhalb derselben Verbindung keine doppelte Notification erzeugt.

```
{
  "type": "notification",
  "messageId": "7c4d2f8a-1b5e-4d47-a9c0-d2f8a1b5e6d4",
  "correlationId": null,
  "timestamp": "2026-07-12T10:15:00Z",
  "payload": {
    "eventType": "statusChanged",
    "providerId": "9910123400002",
    "apiId": "H2API00001",
    "majorVersion": 1,
    "revision": 18,
    "entry": { "...": "Verzeichniseintrag gemäß Schema der Web-API" }
  }
}
```

Eigenschaft	Typ / Format	Pflicht	Beschreibung
eventType	string (enum)	MUSS	Ereignisart gemäß Kapitel 6.2.
providerId	string	MUSS	MP-ID des verantwortlichen Marktpartners.
apiId	string	MUSS	API-Kennung des betroffenen Verzeichniseintrags.
majorVersion	integer (int32)	MUSS	Hauptversion des betroffenen API-Webdienstes.
revision	integer	MUSS	Revision im Bereich 1 bis 9.007.199.254.740.991. Bei Änderungen ist dies die Revision nach dem Ereignis; bei subscriptionEnded die aktuelle Revision ohne Erhöhung.

Eigenschaft	Typ / Format	Pflicht	Beschreibung
entry	object null	MUSS	Vollständiger Verzeichniseintrag gemäß Schema der synchronen Web-API. Bei entryDeleted und subscriptionEnded MUSS der Wert null sein; bei den übrigen bekannten Ereignisarten MUSS ein Objekt übertragen werden.
reason	string	BEDINGT	Bei subscriptionEnded verpflichtender maschinenlesbarer lowerCamelCase-Code, z. B. serverMaintenance oder serverCapacity. Bei anderen bekannten Ereignisarten DARF die Eigenschaft NICHT enthalten sein.

6.2 Ereignisarten

eventType	Bedeutung
entryUpserted	Ein Verzeichniseintrag wurde neu angelegt oder aktualisiert (Teil 2, Kapitel 5.4.2 lit. a [API-GL]).
entryDeleted	Ein Verzeichniseintrag wurde endgültig gelöscht (lit. b). Die Deaktivierung eines Eintrags ist keine Löschung; sie wird als statusChanged mit vollständigem Eintrag übertragen.
statusChanged	Der Lifecycle-Status oder Betriebsstatus eines Verzeichniseintrags hat sich geändert (lit. c); dies schließt automatische Statusänderungen gemäß Kapitel 7.4 sowie die Deaktivierung eines Eintrags ein.
serviceInfoChanged	Technische Service-Informationen, insbesondere Version, Kontakt oder availabilityMonitoringStatus, haben sich geändert (lit. d).
subscriptionEnded	Eine zuvor aktive Subscription wurde serverseitig beendet, z. B. aufgrund einer Serrrichtlinie oder aus betrieblichen Gründen. Eine abgelehnte Einrichtung wird mit error, eine clientseitige Kündigung mit unsubscribeAck beantwortet.

Für den reason-Code der eventType = subscriptionEnded (Kapitel 6.1) sind die folgenden Werte definiert:

reason	Bedeutung
serverMaintenance	Beendigung wegen geplanter Wartung des Verzeichnisdienstes.
serverCapacity	Beendigung aus Kapazitäts- oder Lastgründen.

reason	Bedeutung
serverPolicyOther	Beendigung aus sonstigen betrieblichen oder richtlinienbedingten Gründen.

MUST Neue reason-Codes werden durch die API-Governance im Rahmen des Änderungsmanagements gemäß Teil 1, Kapitel 4.2 [API-GL] eingeführt und in dieser Tabelle geführt. Eine neue Ausprägung ist eine abwärtskompatible Änderung im Sinne von Kapitel 2.3.

MUST Clients MÜSSEN einen reason-Wert, der nicht in dieser Tabelle geführt wird, aber dem Muster gemäß Kapitel 11.2 entspricht, als sonstigen serverseitigen Grund tolerieren und DÜRFEN dies NICHT als Protokollfehler behandeln.

MUST Für jede abgeschlossene Änderung eines Verzeichniseintrags und die dadurch erzeugte neue Revision MUSS der Server genau eine Änderungs-notification an jede betroffene aktive Subscription der Verbindung senden. Erfüllt eine Änderung mehrere Ereignisarten, MUSS die spezifischste Ereignisart mit der Priorität `entryDeleted` vor `statusChanged` vor `serviceInfoChanged` vor `entryUpserted` verwendet werden. Eine notification mit `eventType = subscriptionEnded` ist keine Änderung des Verzeichniseintrags und fällt nicht unter diese Einmaligkeitsregel.

MUST Mit dem Versand einer notification mit `eventType = entryDeleted` endet die betroffene Subscription automatisch. Eine zusätzliche notification mit `eventType = subscriptionEnded` DARF in diesem Fall NICHT gesendet werden.

MUST Eine notification mit `eventType = subscriptionEnded` DARF nur für eine zuvor aktive, serverseitig beendete Subscription verwendet werden. Sie ist die letzte Nachricht für diese Subscription; nach ihrem Versand DÜRFEN keine weiteren Benachrichtigungen für die Subscription gesendet werden.

MUST Wird ein Verzeichniseintrag nach einer Löschung unter demselben Schlüssel `providerId/apiId/majorVersion` erneut angelegt, MUSS die Revision streng monoton oberhalb der letzten vor der Löschung vergebenen Revision fortgeführt werden. Ein Zurücksetzen der Revision DARF NICHT erfolgen.

MUST Der Verzeichnisdienst MUSS je Schlüssel `providerId/apiId/majorVersion` die zuletzt vergebene Revision dauerhaft als Revisions-Hochwassermarke vorhalten, auch über eine Löschung des Verzeichniseintrags hinaus. Diese Vorhaltung MUSS auf die für die strenge Monotonie erforderlichen Angaben (Schlüssel und zuletzt vergebene Revision) beschränkt bleiben und ermöglicht die monotone Fortführung der Revision bei einer späteren Wiederanlage.

MUST Eine Wiederanlage nach vorheriger Löschung wird an bestehende Subscriptions nicht rückwirkend nachgeliefert; die mit dem Versand von `entryDeleted` automatisch beendete Subscription (Kapitel 6.1) lebt hierdurch nicht wieder auf. Ein Client, der eine Wiederanlage unter demselben Schlüssel feststellen will, MUSS den Eintrag erneut abonnieren oder den aktuellen Stand über die synchrone Web-API abrufen.

Weitergehende Angaben zu einer Löschung (insbesondere Löszeitpunkt, löschende Stelle und Audit-Referenz) unterliegen den allgemeinen Aufbewahrungs- und Löschfristen und sind nicht Teil der dauerhaft vorzuhaltenden Revisions-Hochwassermarke.

6.3 Umfang der Benachrichtigungen

MUST Benachrichtigungen MÜSSEN den betroffenen Verzeichniseintrag vollständig in `entry` enthalten, mit Ausnahme von `entryDeleted` und `subscriptionEnded`. Die Deaktivierung eines weiterhin vorhandenen Eintrags wird als `statusChanged` mit vollständigem `entry` übertragen. Der Aufbau des Eintrags entspricht dem jeweils gültigen Schema der synchronen Web-API; die Größenbegrenzung gemäß Kapitel 4.1 gilt.

MUST Der Server MUSS sicherstellen, dass er nur über Einträge informiert, für die eine aktive Subscription besteht. Beendet der Server eine aktive Subscription aus betrieblichen Gründen (z. B. Serverrichtlinie gemäß Kapitel 6.2), MUSS er eine abschließende `notification` mit `eventType = subscriptionEnded`, `entry = null` und einem `reason-Code` senden; weitere Inhalte DÜRFEN NICHT übermittelt werden.

6.4 Reihenfolge, Duplikate und Verspätungen

MUST Der Server MUSS Benachrichtigungen zu demselben Verzeichniseintrag innerhalb einer Verbindung in der Reihenfolge aufsteigender Revisionen versenden.

MUST Der Outbox-Dispatcher MUSS Ereignisse desselben Verzeichnisschlüssels in aufsteigender `revision` verarbeiten. Ein Ereignis mit einer höheren Revision DARF für denselben Schlüssel nicht vor einem noch ausstehenden Ereignis mit niedrigerer Revision an dieselbe Verbindung übergeben werden.

MUST Für Benachrichtigungen zu unterschiedlichen Verzeichniseinträgen besteht keine globale Reihenfolgegarantie. Clients MÜSSEN den Zustand je Schlüssel `providerId/apiId/major-Version` getrennt führen.

SHOULD Clients SOLLTEN empfangene Benachrichtigungen so verarbeiten, dass mehrfache oder verspätete Benachrichtigungen nicht zu einem fehlerhaften lokalen Zustand führen. Hierzu ist die Revisionskennung gemäß Kapitel 4.4.2 auszuwerten.

MUST Empfängt ein Client eine `notification` mit einer unbekannten `eventType` und einer `revision` oberhalb seines lokalen Stands, MUSS er den betroffenen Verzeichniseintrag über die synchrone Web-API resynchronisieren. Die unbekannte Ereignisart DARF weder zum Abbruch der Verbindung noch zu einer unmittelbaren lokalen Zustandsänderung führen.

7. Technische Verfügbarkeitsmeldung

Der Verzeichnisdienst unterstützt technische Verfügbarkeitsmeldungen über die WebSocket-API gemäß Teil 2, Kapitel 5.8 der API-Guideline [API-GL]. Eine technische Verfügbarkeitsmeldung dient der Anzeige, dass ein bestimmter marktpartnerbezogener API-Webdienst durch das verantwortliche System des Marktpartners als technisch verfügbar gemeldet wird. Sie ist von einer Subscription strikt zu unterscheiden.

MUST Eine technische Verfügbarkeitsmeldung DARF nur für Verzeichniseinträge erfolgen, für die der authentifizierte Marktpartner verantwortlich und berechtigt ist. Die `providerId` der Meldung MUSS mit der MP-ID aus dem Client-Zertifikat übereinstimmen.

MUST NOT Eine technische Verfügbarkeitsmeldung DARF NICHT für Verzeichniseinträge fremder MP-IDs erfolgen (Fehlercode ERR-AVL-001). Eine reine Subscription DARF NICHT als technische Verfügbarkeitsmeldung behandelt werden; ebenso DÜRFEN reine Abfragen, reine WebSocket-Verbindungen oder technische Ping-/Pong-Frames NICHT zu einer Änderung des Betriebsstatus führen.

7.1 Beginn der Verfügbarkeitsmeldung (`availabilityStart`)

```
{
  "type": "availabilityStart",
  "messageId": "e6d47c4d-2f8a-4b5e-9c0d-1b5e6d47a9c0",
  "correlationId": null,
  "timestamp": "2026-07-12T09:31:00Z",
  "signature": "eyJhbGciOiJI1cm46aDI6andzOmFsZzpicmFpbmBvb2xQMjU2cjEtc2hhMjU2IiwidHl-
wIjoiaDItd3Mtc2lnK2p3cyIsImI2NCI6ZmFsc2UsImNyaXQi-
OlsiYjY0Il0sImtpZCI6Im5TbFkwblo4LWV4YW1wbGUifQ..gekuerzt",
  "payload": {
    "providerId": "9910567800001",
    "apiId": "H2API00001",
    "majorVersion": 1
  }
}
```

MUST Der Server MUSS eine zulässige `availabilityStart`-Nachricht mit `availabilityAck` bestätigen. Die Verfügbarkeitsmeldung ist ab Versand der Bestätigung aktiv.

MUST Je Verzeichniseintrag und Verbindung DARF höchstens eine aktive Verfügbarkeitsmeldung bestehen. Eine erneute `availabilityStart`-Nachricht für denselben Schlüssel innerhalb derselben Verbindung MUSS idempotent behandelt, als Erneuerung des Empfangszeitpunkts gewertet und mit `availabilityAck` beantwortet werden.

MUST Die `availabilityAck` MUSS den Schlüssel des betroffenen Verzeichniseintrags sowie `operation` enthalten. Für `operation = start` oder `heartbeat` MUSS `validUntil` den vom Server berechneten Zeitpunkt enthalten, bis zu dem die Verfügbarkeitsmeldung ohne weiteren

Heartbeat aktiv bleibt. Für `operation = stop` DARF `validUntil` NICHT enthalten sein (Kapitel 11.4).

MUST Je WebSocket-Verbindung DÜRFEN höchstens 200 technische Verfügbarkeitsmeldungen gleichzeitig aktiv sein. Darüber hinausgehende `availabilityStart`-Nachrichten MUSS der Server mit `ERR-AVL-003` ablehnen. Eine idempotente Wiederholung für eine bereits aktive Verfügbarkeitsmeldung legt keine zusätzliche Meldung an und DARF wegen des ausgeschöpften Limits NICHT abgelehnt werden.

7.2 Fachlicher Heartbeat (`availabilityHeartbeat`)

MUST Zur Aufrechterhaltung einer aktiven Verfügbarkeitsmeldung MUSS der Client zyklisch eine `availabilityHeartbeat`-Nachricht mit demselben Payload-Schema wie `availabilityStart` senden. Das Heartbeat-Intervall beträgt 60 Sekunden (Kapitel 10).

MUST Verwaltet ein Client mehrere aktive Verfügbarkeitsmeldungen, MUSS er deren Heartbeats gleichmäßig über das 60-Sekunden-Intervall verteilen. Synchronisierte Heartbeat-Bursts, die das Nachrichtenratenlimit überschreiten können, DÜRFEN NICHT erzeugt werden.

MUST Für `Heartbeat-Timeout` und `validUntil` ist ausschließlich der Empfangszeitpunkt beim Server maßgeblich. Der `timestamp` der `availabilityHeartbeat`-Nachricht DARF dafür NICHT verwendet werden.

MUST Der Server MUSS den Empfang jedes zulässigen Heartbeats mit `availabilityAck` bestätigen. `validUntil` MUSS dem serverseitigen Empfangszeitpunkt zuzüglich 180 Sekunden entsprechen.

MUST Bleibt der Heartbeat länger als das `Heartbeat-Timeout` von 180 Sekunden aus, gilt die Verfügbarkeitsmeldung als weggefallen (Kapitel 7.4).

Technische WebSocket-Ping-/Pong-Frames sind hiervon zu unterscheiden. Sie dienen ausschließlich der Erkennung und Aufrechterhaltung der Verbindung und ersetzen keinen fachlichen Heartbeat (Teil 2, Kapitel 5.8.1 [API-GL]).

7.3 Beendigung der Verfügbarkeitsmeldung (`availabilityStop`)

Aufbau der Nachricht `availabilityStop`:

```
{
  "type": "availabilityStop",
  "messageId": "f7e58d6c-3a4b-4c5d-8e9f-0a1b2c3d4e5f",
  "correlationId": null,
  "timestamp": "2026-07-12T10:00:00Z",
```

```
"signature": "eyJhbGciOiJI1cm46aDI6andzOmFsZzpic-  
mFpbnBvb2xQMjU2cjEtc2hhMjU2IiwidHlwIjoiaDItd3Mtc2lnK2p3cy-  
IsImI2NCI6ZmFsc2UsImNyaXQiOlsiYjY0Il0sImtpZCI6Im5TbFkwblo4LWV4YW1wbGU-  
ifQ..gekuerzt",  
"payload": {  
  "providerId": "9910567800001",  
  "apiId": "H2API00001",  
  "majorVersion": 1,  
  "reason": "serviceUnavailable"  
}  
}
```

MUST Die ordnungsgemäße Beendigung einer Verfügbarkeitsmeldung MUSS durch `availabilityStop` erfolgen. Der Payload MUSS zusätzlich zum Verzeichnisschlüssel die Eigenschaft `reason` enthalten. Der Server MUSS die Beendigung mit `availabilityAck` bestätigen.

Für `reason` sind ausschließlich die folgenden Werte zulässig:

- `serviceUnavailable`: Der API-Webdienst ist ausdrücklich als technisch nicht verfügbar gemeldet.
- `clientShutdown`: Der Monitoring-Client wird beendet; hieraus folgt keine unmittelbare Aussage über den API-Webdienst.
- `connectionMigration`: Die Verfügbarkeitsmeldung wird auf eine andere Verbindung verlagert.
- `plannedRestart`: Der Monitoring-Client oder seine Verbindung wird geplant neu gestartet.
- `other`: Sonstige Beendigung ohne ausdrückliche Aussage, dass der API-Webdienst nicht verfügbar ist.

MUST Nur `reason = serviceUnavailable` DARF unmittelbar als endpunktspezifische negative Evidenz bewertet werden. Die übrigen Gründe MÜSSEN zunächst ausschließlich zur Beendigung der Verfügbarkeitsmeldung und zum Zustand `unconfirmed` führen.

Die Karenzzeit gemäß Kapitel 10 gilt auch bei `reason = serviceUnavailable`. Sie eröffnet einem redundanten System des Marktpartners die Möglichkeit, innerhalb der Karenzzeit eine neue gültige Verfügbarkeitsmeldung aufzubauen, bevor der `operationalStatus` automatisch geändert wird.

MUST Geplante Wartung des API-Webdienstes MUSS über den manuellen Override einschließlich `maintenanceWindow` gemäß Kapitel 7.7 gepflegt werden und DARF NICHT allein durch `availabilityStop` modelliert werden.

MUST NOT Die Beendigung einer nicht bestehenden Verfügbarkeitsmeldung DARF NICHT als Fehler behandelt werden. Der Server MUSS auch in diesem Fall mit `availabilityAck` und `operation = stop` antworten.

7.4 Wegfall der Verfügbarkeitsmeldung und Statusbehandlung

Ein Wegfall der Verfügbarkeitsmeldung liegt insbesondere vor bei:

- ordnungsgemäßer Beendigung durch `availabilityStop`,
- Abbruch der WebSocket-Verbindung,
- Ausbleiben des fachlichen Heartbeats über das Heartbeat-Timeout hinaus oder
- serverseitiger Beendigung der Verbindung.

MUST Besteht für denselben Verzeichniseintrag mindestens eine weitere gültige Verfügbarkeitsmeldung, MUSS `availabilityMonitoringStatus.state confirmed` bleiben. Eine Karenzzeit oder automatische Statusänderung DARF in diesem Fall NICHT ausgelöst werden.

MUST Besteht keine weitere gültige Verfügbarkeitsmeldung, MUSS der Verzeichnisdienst den Wegfall als endpunktspezifischen Wegfall, neutrale Beendigung oder Störung der Überwachungsinfrastruktur klassifizieren.

Ein endpunktspezifischer Wegfall liegt insbesondere vor bei:

- `availabilityStop` mit `reason = serviceUnavailable`,
- isoliertem Ausbleiben des fachlichen Heartbeats, während der WebSocket-Dienst und seine abhängigen Infrastrukturkomponenten ordnungsgemäß arbeiten,
- isoliertem Verlust einer Clientverbindung, sofern keine Hinweise auf eine zentrale oder korrelierte Störung vorliegen, oder
- einer zusätzlich vorgesehenen, fehlgeschlagenen aktiven technischen Prüfung des konkreten Endpunkts.

Eine neutrale Beendigung liegt bei `availabilityStop` mit `clientShutdown`, `connectionMigration`, `plannedRestart` oder `other` sowie bei einer planmäßigen Verbindungsrotation mit Close-Code 4005 vor.

Eine Störung der Überwachungsinfrastruktur liegt insbesondere vor bei:

- geplanter serverseitiger Beendigung mit Close-Code 1001,
- serverseitigem Fehler mit Close-Code 1011 oder Sendepufferüberlauf mit Close-Code 4004,
- festgestellter Störung des WebSocket-Dienstes, seiner Datenhaltung, des Clusters, der PKI-, DNS- oder Netzabhängigkeiten oder
- Auslösung des Mass-Loss-Circuit-Breakers.

MUST Bei einem endpunktspezifischen Wegfall MUSS `availabilityMonitoringStatus.state` unverzüglich auf `unconfirmed` gesetzt werden. Bei Heartbeat-Ausfall gilt der Wegfall 180 Sekunden nach dem letzten empfangenen Heartbeat als festgestellt; bei `availabilityStop` mit `serviceUnavailable` und bei isoliertem Verbindungsverlust gilt er unmittelbar als festgestellt.

MUST Besteht nach Ablauf der Karenzzeit von 120 Sekunden weiterhin keine gültige Verfügbarkeitsmeldung, MUSS der Verzeichnisdienst den `operationalStatus` auf `down` setzen, sofern `availabilityMonitoring.enabled = true`, kein manueller Override aktiv und der `life-cycleStatus` nicht `retired` ist.

MUST Bei einer neutralen Beendigung MUSS `availabilityMonitoringStatus.state` auf `unconfirmed` gesetzt und eine neue vollständige Karenzzeit von 120 Sekunden begonnen werden. Der `operationalStatus` DARF nicht unmittelbar geändert werden. Besteht nach Ablauf der Karenzzeit weiterhin keine gültige Verfügbarkeitsmeldung und arbeitet die Überwachungsinfrastruktur ordnungsgemäß, MUSS der fortdauernde Wegfall als endpunktspezifisch behandelt und der `operationalStatus` unter den Voraussetzungen dieses Kapitels auf `down` gesetzt werden.

MUST Bei einer Störung der Überwachungsinfrastruktur MUSS `availabilityMonitoringStatus.state` auf `monitoringUnavailable` gesetzt werden. Der `operationalStatus` DARF aufgrund des Wegfalls der Verfügbarkeitsmeldungen NICHT geändert werden; laufende Heartbeat-, Timeout- und Karenzfristen MÜSSEN ausgesetzt werden.

MUST Solange der Mass-Loss-Circuit-Breaker geöffnet ist beziehungsweise die Störung der Überwachungsinfrastruktur andauert, MUSS `availabilityMonitoringStatus.state = monitoringUnavailable` bestehen bleiben. Ein einzelner Verzeichniseintrag DARF diesen Zustand NICHT allein aufgrund einer neuen bestätigten Verfügbarkeitsmeldung verlassen; die Neubewertung je Eintrag erfolgt ausschließlich mit der Rücksetzung des Circuit-Breakers gemäß den nachfolgenden Absätzen.

MUST Der Verzeichnisdienst MUSS je Umgebung einen clusterweit konsistenten Mass-Loss-Circuit-Breaker implementieren. Er MUSS öffnen, wenn innerhalb eines Beobachtungsfensters von 60 Sekunden mindestens 20 UND mindestens 10 Prozent der zu Beginn dieses Beobachtungsfensters aktiven Verfügbarkeitsmeldungen aufgrund derselben oder einer technisch korrelierten Fehlerursache wegfallen oder wenn der interne Gesundheitszustand der Überwachungsinfrastruktur nicht `operational` ist.

MUST Solange der Circuit-Breaker geöffnet ist, DÜRFEN keine automatischen `down`-Übergänge aufgrund fehlender WebSocket-Verfügbarkeitsmeldungen ausgeführt werden.

MUST Der Circuit-Breaker DARF erst zurückgesetzt werden, nachdem die Überwachungsinfrastruktur mindestens 300 Sekunden stabil als `operational` bewertet wurde.

MUST Nach der Rücksetzung MUSS `availabilityMonitoringStatus.state` für Einträge mit mindestens einer gültigen Verfügbarkeitsmeldung unmittelbar auf `confirmed` gesetzt werden; für diese Einträge DARF keine Karenzzeit beginnen. Für Einträge ohne gültige Verfügbarkeitsmeldung MUSS `availabilityMonitoringStatus.state` auf `unconfirmed` gesetzt und eine neue vollständige Karenzzeit von 120 Sekunden begonnen werden. Während der Störung abgelaufene Fristen DÜRFEN NICHT rückwirkend berücksichtigt werden. Läuft die neue Karenzzeit bei ordnungsgemäß arbeitender Überwachungsinfrastruktur ohne neue gültige Meldung ab, MUSS der fortdauernde Wegfall als endpunktspezifisch behandelt werden.

MUST NOT Für Verzeichniseinträge, für die die Verfügbarkeitsüberwachung nicht aktiviert ist, DARF der Betriebsstatus NICHT automatisch geändert werden.

7.5 Wiederaufnahme der Verfügbarkeit

MUST Wird nach einem Wegfall eine Verfügbarkeitsmeldung aufgebaut und bestätigt, MUSS der Verzeichnisdienst `availabilityMonitoringStatus.state` auf `confirmed` setzen und laufende Karenz- oder Wiederanlaufzeiten für den Eintrag aufheben, sofern nicht der Zustand `monitoringUnavailable` gemäß Kapitel 7.4 besteht; in diesem Fall erfolgt die Neubewertung ausschließlich mit der Rücksetzung des Circuit-Breakers.

MUST Einen zuvor durch den WebSocket-Mechanismus automatisch auf `down` gesetzten `operationalStatus` MUSS der Verzeichnisdienst wieder auf `operational` setzen, sofern die automatische Statussteuerung gemäß Kapitel 7.7 aktiv ist, kein manueller Override gesetzt ist und der `lifecycleStatus` nicht `retired` ist.

MUST Nach einem Zustand `monitoringUnavailable` DARF die Wiederherstellung der Überwachungsinfrastruktur NICHT rückwirkend zu `down` führen. Maßgeblich sind ausschließlich neue Verfügbarkeitsmeldungen und neu beginnende Fristen nach Wiederherstellung beziehungsweise Rücksetzung des Circuit-Breakers.

MUST Ist ein ausdrücklicher manueller Override des Betriebsstatus aktiv, DARF die WebSocket-Verfügbarkeitsüberwachung den `operationalStatus` NICHT verändern. Die Gültigkeit eingehender Verfügbarkeitsmeldungen und `availabilityMonitoringStatus` MÜSSEN dennoch weitergeführt werden.

7.6 Nachvollziehbarkeit automatischer Statusänderungen

MUST Automatische Statusänderungen und Änderungen des `availabilityMonitoringStatus` durch den Verzeichnisdienst MÜSSEN nachvollziehbar protokolliert werden. Zu protokollieren sind mindestens betroffener Verzeichniseintrag, vorheriger und neuer Zustand, Zeitpunkt, Grund, Art der Feststellung und auslösender Mechanismus.

MUST Öffnung und Rücksetzung des Mass-Loss-Circuit-Breakers MÜSSEN zusätzlich mit Ursache, Auslösewerten, Anzahl betroffener Verfügbarkeitsmeldungen und betroffener Einträge revisions-sicher protokolliert und betrieblich alarmiert werden.

MUST Ändert sich ausschließlich `availabilityMonitoringStatus.state`, MUSS genau eine Revision erzeugt und eine `notification` mit `eventType = serviceInfoChanged` gesendet werden.

MUST Ändern sich `availabilityMonitoringStatus.state` und `operationalStatus` im selben atomaren Vorgang, MUSS genau eine Revision erzeugt und eine `notification` mit `eventType = statusChanged` gesendet werden.

MUST Der Empfang eines Heartbeats im bereits bestehenden Zustand `confirmed` DARF keine Revision und keine `notification` erzeugen.

Übersicht der Zustandsübergänge von `availabilityMonitoringStatus.state` (informativ):

`inactive` → `unconfirmed`: `enabled = true` ohne bestätigte `availabilityStart`-Nachricht (`reason monitoringEnabledAwaitingStart`); keine Down-Karenz. `unconfirmed` → `confirmed`: erste bestätigte `availabilityStart`-Nachricht; Aktivierung der automatischen Statussteuerung (`reason availabilityConfirmed`).

`confirmed` → `unconfirmed`: endpunktspezifischer Wegfall (`heartbeatExpired`, `connectionLost`, `serviceUnavailableReported`) oder neutrale Beendigung; Beginn der Karenzzeit von 120 Sekunden.

`unconfirmed` → `confirmed`: neue bestätigte Verfügbarkeitsmeldung; laufende Karenz- und Wiederanlauffristen werden aufgehoben.

`unconfirmed` (Karenzablauf): Ablauf der Karenzzeit ohne gültige Meldung bei ordnungsgemäß arbeitender Überwachungsinfrastruktur; der `operationalStatus` wird unter den Voraussetzungen des Kapitels 7.4 auf `down` gesetzt, `state` bleibt `unconfirmed`.

beliebig → `monitoringUnavailable`: Störung der Überwachungsinfrastruktur oder geöffneter Mass-Loss-Circuit-Breaker (`reason monitoringInfrastructureUnavailable`); laufende Fristen werden ausgesetzt, der `operationalStatus` bleibt unverändert.

`monitoringUnavailable` → `confirmed` oder `unconfirmed`: Rücksetzung des Circuit-Breakers (`reason monitoringInfrastructureRestored`); `confirmed` bei mindestens einer gültigen Meldung, andernfalls `unconfirmed` mit neuer vollständiger Karenzzeit.

beliebig → `inactive`: Deaktivierung der Überwachung (`reason monitoringDisabled`) oder Wechsel des `lifecycleStatus` nach `retired` (`reason lifecycleRetired`); keine Karenzzeit.

7.7 Aktivierung der Verfügbarkeitsüberwachung im Verzeichniseintrag

Die Nutzung der technischen Verfügbarkeitsmeldung wird je Verzeichniseintrag über das marktpartnerseitig gepflegte Objekt `availabilityMonitoring` gesteuert:

```
"availabilityMonitoring": {  
  "enabled": true,  
  "mechanism": "websocketHeartbeat"  
}
```

Der Verzeichnisdienst führt zusätzlich das systemseitige Objekt `availabilityMonitoringStatus`:

```
"availabilityMonitoringStatus": {
```

```
"state": "confirmed",  
"changedAt": "2026-07-12T09:31:01Z",  
"reason": "availabilityConfirmed"  
}
```

MUST Der Verzeichniseintrag MUSS das geschlossene Objekt `availabilityMonitoring` mit `enabled` (boolean) und `mechanism = websocketHeartbeat` enthalten. Bei initialer Anlage ist `enabled = false` zu verwenden, sofern die Überwachung nicht ausdrücklich aktiviert wird. Bei `enabled = false` ist `availabilityMonitoringStatus.state = inactive` (reason `monitoringDisabled`); dies ist der Ausgangszustand eines neu angelegten Verzeichniseintrags ohne aktivierte Überwachung. Bei `enabled = true` ohne bestätigte `availabilityStart`-Nachricht MÜSSEN `availabilityMonitoringStatus.state = unconfirmed` und `reason = monitoringEnabledAwaitingStart` gelten. Allein aufgrund des Ausbleibens der erstmaligen `availabilityStart`-Nachricht DARF keine Karenzzeit für einen automatischen Übergang des `operationalStatus` auf `down` beginnen. Die automatische Statussteuerung durch `websocketHeartbeat` wird erst mit der ersten bestätigten `availabilityStart`-Nachricht aktiviert.

MUST Das Objekt `availabilityMonitoring` DARF nur durch den für den Verzeichniseintrag verantwortlichen Marktpartner über den Pflegeprozess gemäß Teil 2, Kapitel 4.7 [API-GL] geändert werden. Soweit Verzeichniseinträge signiert werden, MUSS `availabilityMonitoring` zum marktpartnerseitig gepflegten und signierten Teil des Eintrags gehören.

MUST `availabilityMonitoringStatus` MUSS durch den Verzeichnisdienst geführt werden und DARF NICHT vom Marktpartner geändert oder in die Marktpartnersignatur einbezogen werden.

MUST `availabilityMonitoringStatus.state` MUSS einen der Werte `inactive`, `confirmed`, `unconfirmed` oder `monitoringUnavailable` enthalten. `changedAt` MUSS den Zeitpunkt des letzten Zustandswechsels in UTC enthalten. `reason` MUSS ein maschinenlesbarer lower-CamelCase-Code sein.

Für Version 1 sind insbesondere die Gründe `monitoringDisabled`, `monitoringEnabledAwaitingStart`, `availabilityConfirmed`, `serviceUnavailableReported`, `heartbeatExpired`, `connectionLost`, `monitoringInfrastructureUnavailable`, `monitoringInfrastructureRestored` und `lifecycleRetired` vorgesehen. Weitere Gründe DÜRFEN im Rahmen des Änderungsmanagements ergänzt werden.

MUST Jede Änderung von `availabilityMonitoring` MUSS die Revision des Verzeichniseintrags erhöhen und eine `notification` mit `eventType = serviceInfoChanged` auslösen.

MUST Der Verzeichnisdienst MUSS zusätzlich einen internen Steuerungszustand für die Herkunft des Betriebsstatus führen. Ein manueller Betriebsstatus hat nur dann Vorrang vor der Automatik, wenn er im Pflegeprozess ausdrücklich als manueller Override gekennzeichnet wurde.

MUST Ist `enabled = true` und kein manueller Override aktiv, aktiviert eine bestätigte `availabilityStart`-Nachricht die automatische Steuerung durch `websocketHeartbeat`. Der Verzeichnisdienst MUSS `availabilityMonitoringStatus.state` auf `confirmed` und den

`operationalStatus` auf `operational` setzen, sofern dieser nicht bereits `operational` ist. Laufende Karenzzeiten MÜSSEN aufgehoben werden.

MUST Ein manueller Override DARF ausschließlich durch einen hierfür berechtigten Marktpartner oder die API-Governance über den synchronen Pflegeprozess gesetzt oder aufgehoben werden. Der Override MUSS mindestens den gesetzten `operationalStatus`, einen maschinenlesbaren Grund, den Zeitpunkt, die verantwortliche Stelle und optional `validUntil` enthalten; eine Änderung über die WebSocket-Verbindung ist unzulässig.

MUST Nach Aufhebung oder Ablauf eines Overrides MUSS der Server den Betriebsstatus anhand der aktuell gültigen Verfügbarkeitsmeldungen und des aktuellen `availabilityMonitoringStatus` neu bewerten. Bei `monitoringUnavailable` DARF keine automatische Verschlechterung erfolgen.

MUST Wird durch einen manuellen Override der `operationalStatus` `maintenance` gesetzt, MUSS der Override ein `maintenanceWindow` mit Start- und Endzeitpunkt gemäß Teil 2, Kapitel 4.8.2 [API-GL] enthalten. `validUntil` MUSS in diesem Fall enthalten sein und exakt dem Endzeitpunkt des `maintenanceWindow` entsprechen. Mit Erreichen dieses Zeitpunkts MUSS der Override automatisch enden und der Betriebsstatus gemäß diesem Kapitel neu bewertet werden.

MUST Wird durch einen manuellen Override der `operationalStatus` `degraded` gesetzt, MUSS die Art der Einschränkung im Override dokumentiert sein.

MUST Wird `availabilityMonitoring` deaktiviert, MUSS der Server bestehende Verfügbarkeitsmeldungen für den Eintrag beenden, `availabilityMonitoringStatus.state` auf `inactive` setzen und den `operationalStatus` unverändert lassen. Nachfolgende `availabilityStart`- und `availabilityHeartbeat`-Nachrichten MUSS er mit ERR-AVL-001 ablehnen.

MUST NOT Für Verzeichniseinträge mit `lifecycleStatus` = `retired` DARF der Betriebsstatus NICHT automatisch geändert werden; er beträgt dauerhaft `down`.

MUST `availabilityStart`- und `availabilityHeartbeat`-Nachrichten für Verzeichniseinträge mit `lifecycleStatus` = `retired` MUSS der Server mit ERR-AVL-001 ablehnen. Wechselt ein Eintrag mit aktiver Verfügbarkeitsmeldung in `retired`, MUSS der Server die Meldung beenden und `availabilityMonitoringStatus.state` auf `inactive` mit `reason` = `lifecycleRetired` setzen, ohne eine Karenzzeit auszulösen.

8. Fehlerbehandlung

8.1 Fehlernachricht (error)

MUST Treten bei der Verarbeitung von Client-Nachrichten oder von Ereignissen Fehler auf, MUSS der Server den Client mit einer error-Nachricht informieren. Die Fehlernachricht MUSS strukturierte Informationen über den Fehler enthalten und über die `correlationId` erkennen lassen, auf welche Anfrage oder welchen Sachverhalt sich der Fehler bezieht.

```
{
  "type": "error",
  "messageId": "a9c0d2f8-6d47-4c4d-8a1b-5e6d47a9c0d2",
  "correlationId": "3f2a6c1e-9d4b-4c1a-8e2f-5b7d9a0c3e11",
  "timestamp": "2026-07-12T09:30:02Z",
  "payload": {
    "code": "ERR-SUB-001",
    "title": "Unbekannter Verzeichniseintrag",
    "detail": "Zu providerId/apiId/majorVersion existiert kein Verzeichniseintrag.",
    "context": {
      "providerId": "9910123400002",
      "apiId": "H2API00001",
      "majorVersion": 2
    }
  }
}
```

Eigenschaft	Typ / Format	Pflicht	Beschreibung
code	string (enum)	MUSS	Fehlercode gemäß Kapitel 8.2.
title	string	MUSS	Kurze, sprachlich neutrale Beschreibung der Fehlerklasse.
detail	string	OPTIONAL	Ergänzende Beschreibung des konkreten Fehlers. DARF keine schutzbedürftigen Informationen enthalten.
context	object	OPTIONAL	Strukturierte Angaben zum betroffenen Sachverhalt (z. B. Kennungen des Verzeichniseintrags).

MUST NOT Eine Fehlernachricht DARF den Zustand bestehender Subscriptions und Verfügbarkeitsmeldungen NICHT verändern.

MAY Im Falle kritischer Fehler, bei denen der korrekte Betrieb der Verbindung nicht mehr gewährleistet werden kann, DARF der Server die WebSocket-Verbindung mit einem Close-Code gemäß Kapitel 8.3 beenden.

8.2 Fehlercodes

Fehlercode	Bedeutung	Verhalten des Clients
ERR-MSG-001	Unbekannter, nicht unterstützter oder in dieser Richtung unzulässiger Nachrichtentyp.	Nachricht korrigieren; keine Wiederholung ohne Änderung.
ERR-MSG-002	messageId wurde innerhalb derselben Verbindung bereits verwendet.	Neue messageId vergeben; ursprüngliche Nachricht nicht unverändert wiederholen.
ERR-VAL-001	Verletzung des Nachrichtenschemas, z. B. Struktur, Datentyp, Format oder Pflichtfeld.	Nachricht korrigieren; keine Wiederholung ohne Änderung.
ERR-AUTH-001	Fehlende Berechtigung für eine Operation dieser Schnittstelle. Aufgrund des offenen Sichtbarkeitsmodells für Subscriptions (Kapitel 5) wird dieser Code in dieser Version nicht ausgelöst; er ist für künftige Erweiterungen reserviert.	Keine Wiederholung; Berechtigung klären.
ERR-SUB-001	Der referenzierte Verzeichniseintrag existiert für einen grundsätzlich berechtigten Client nicht.	Kennungen prüfen; ggf. über die Web-API verifizieren.
ERR-SUB-002	Zulässige Anzahl aktiver Subscriptions überschritten.	Nicht benötigte Subscriptions kündigen.
ERR-AVL-001	Unzulässige technische Verfügbarkeitsmeldung, z. B. fremde MP-ID, deaktivierte Überwachung oder lifecycleStatus = retired.	Keine Wiederholung; Konfiguration prüfen.
ERR-AVL-002	Heartbeat ohne aktive Verfügbarkeitsmeldung.	Verfügbarkeitsmeldung mit availabilityStart neu aufbauen.
ERR-AVL-003	Zulässige Anzahl aktiver technischer Verfügbarkeitsmeldungen überschritten.	Weitere Verbindung verwenden oder nicht benötigte Meldungen mit availabilityStop beenden.
ERR-SIG-001	Fehlende, strukturell ungültige oder kryptographisch ungültige Nachrichtensignatur oder Verletzung des Signatur-Zeitfensters	Signaturbildung und Systemzeit prüfen; Nachricht mit neuer messageId und neuer Signatur senden.

Fehlercode	Bedeutung	Verhalten des Clients
	gemäß [JWS-SPEC], Kapitel 8.1 (vgl. Kapitel 3.5 dieses Dokuments).	
ERR-RATE-001	Zulässige Nachrichtenrate überschritten.	Senderate reduzieren; Wiederholung erst, wenn das gleitende 10-Sekunden-Fenster die Nachricht wieder zulässt.
ERR-INT-001	Interner Fehler des Verzeichnisdienstes; der Verarbeitungsausgang kann unbekannt sein.	Idempotente Operation mit neuer messageId und Backoff gemäß Kapitel 9.2 wiederholen.

MUST Clients MÜSSEN unbekannte, syntaktisch gültige Fehlercodes als generischen Fehler der korrelierten Operation behandeln. Ein automatischer Retry DARF nur erfolgen, wenn der Fehlercode oder eine spätere kompatible Spezifikation ihn ausdrücklich als wiederholbar kennzeichnet.

8.3 Close-Codes

Für das Schließen der WebSocket-Verbindung werden die folgenden Close-Codes gemäß [RFC6455] sowie anwendungsspezifische Codes im Bereich 4000–4999 verwendet:

Close-Code	Auslöser	Bedeutung
1000	Client / Server	Normale, ordnungsgemäße Beendigung der Verbindung.
1001	Server	Der Server wird beendet oder in Wartung genommen (Going Away). Der Client SOLLTE gemäß Kapitel 9.2 neu verbinden.
1003	Client / Server	Empfang eines Binärframes (Unsupported Data). Die WebSocket-API verwendet ausschließlich Textframes.
1007	Client / Server	Empfangene Nachricht ist kein gültiges UTF-8 (Invalid Frame Payload Data).
1008	Client / Server	Richtlinienvorstoß (Policy Violation), z. B. wiederholte unberechtigte Anfragen.
1009	Client / Server	Empfangene Nachricht überschreitet die maximale Nachrichtengröße (Message Too Big).
1011	Server	Unerwarteter interner Fehler des Servers (Internal Error).
4000	Server	Protokollverletzung auf Anwendungsebene (z. B. wiederholt schemawidrige Nachrichten).
4001	Server	Sicherheitskontext ungültig geworden (z. B. Zertifikat gesperrt oder abgelaufen). Der Client MUSS vor einem erneuten Verbindungsaufbau die Ursache beheben.

Close-Code	Auslöser	Bedeutung
4002	Server	Idle-Timeout: Auf der Verbindung wurden über die festgelegte Dauer (Kapitel 10) weder Nachrichten noch Pong-Frames empfangen.
4003	Server	Wiederholte Überschreitung von Nachrichtenraten oder verbindlichen Ressourcenlimits trotz vorheriger Fehlernachricht.
4004	Server	Überlauf des serverseitigen Sendepuffers (z. B. bei zu langsamer Abnahme durch den Client). Der Client MUSS nach dem Wiederaufbau gemäß Kapitel 9.3 resynchronisieren.
4005	Server	Planmäßige Verbindungsrotation nach Erreichen der maximalen Verbindungsdauer. Der Client MUSS mit neuer H2-Transaction-Id beziehungsweise <code>connectionId</code> und Jitter gemäß Kapitel 9.2 neu verbinden.

MUST Der Server MUSS beim Schließen der Verbindung den zutreffenden Close-Code übertragen. Clients MÜSSEN empfangene Close-Codes auswerten und ihr Wiederanlaufverhalten daran ausrichten (Kapitel 9.2).

8.4 Protokollfehler auf WebSocket-Ebene

MUST Binärframes MUSS der Server mit Close-Code 1003 beantworten. Nachrichten, die kein gültiges UTF-8 sind, MUSS er mit Close-Code 1007 beantworten. Überschreitet eine vollständig zusammengesetzte Nachricht die maximale Nachrichtengröße, MUSS er die Verbindung mit Close-Code 1009 beenden; eine error-Nachricht ist in diesem Fall nicht erforderlich.

MUST Syntaktisch ungültiges JSON MUSS der Server mit `ERR-VAL-001` und `correlationId = null` beantworten. Ist das JSON syntaktisch gültig und enthält es eine gültige `messageId`, verletzt aber das Schema, MUSS `correlationId` diese `messageId` enthalten; andernfalls MUSS `correlationId = null` verwendet werden. Abweichungen, die ausschließlich die Eigenschaft `signature` betreffen, MÜSSEN abweichend hiervon gemäß Kapitel 3.5 mit `ERR-SIG-001` beantwortet werden. Bei wiederholten schema- oder protokollwidrigen Nachrichten (`ERR-VAL-001`) DARF der Server die Verbindung mit Close-Code 4000 beenden. Wiederholte Signaturfehler (`ERR-SIG-001`) werden hiervon abweichend gemäß Kapitel 8.4 mit Close-Code 1008 behandelt.

MUST Unbekannte oder in Client-Richtung unzulässige Nachrichtentypen MUSS der Server mit `ERR-MSG-001` beantworten. Eine doppelte `messageId` MUSS er mit `ERR-MSG-002` beantworten und ohne erneute Verarbeitung verwerfen. Eine `availabilityHeartbeat`-Nachricht ohne aktive Verfügbarkeitsmeldung MUSS mit `ERR-AVL-002` beantwortet werden.

MUST Fragmentierte Nachrichten gemäß [RFC6455] sind zulässig. Die maximale Nachrichten-größe (Kapitel 10) bezieht sich auf die vollständig zusammengesetzte, unkomprimierte UTF-8-Nachricht.

MUST Überschreitet der Client das Nachrichtenratenlimit, MUSS der Server mindestens die erste festgestellte Überschreitung mit `ERR-RATE-001` beantworten. Weitere Nachrichten innerhalb desselben überlasteten Fensters DÜRFEN verworfen werden; bei fortgesetzter Überschreitung DARF der Server mit Close-Code 4003 schließen.

MAY Bei wiederholten Signaturfehlern (`ERR-SIG-001`) innerhalb derselben Verbindung DARF der Server die Verbindung mit Close-Code 1008 beenden.

MUST NOT Die WebSocket-Erweiterung `permessage-deflate` sowie sonstige Kompressionserweiterungen DÜRFEN in Version 1 der Schnittstelle NICHT ausgehandelt oder verwendet werden.

MAY Kann der Server Nachrichten wegen Überlaufs des Sendepuffers nicht zustellen, DARF er die Verbindung mit Close-Code 4004 beenden.

9. Zustell- und Wiederanlaufverhalten

9.1 Zustellgarantien

MUST Die Zustellung von Benachrichtigungen erfolgt nach dem Prinzip *at-most-once*: Der Server MUSS Benachrichtigungen zu Ereignissen, die während einer bestehenden Verbindung mit aktiver Subscription eintreten, an diese Verbindung übergeben. Eine Persistierung, Nachlieferung, Wiederaufnahme (Resume) oder anwendungsseitige Empfangsbestätigung erfolgt NICHT; die Übergabe an die Verbindung belegt nicht die Verarbeitung durch den Client. Revisionen dienen ausschließlich der Resynchronisation gemäß Kapitel 9.3.

MUST Clients MÜSSEN daher davon ausgehen, dass Ereignisse während Verbindungsunterbrechungen verloren gehen können, und den aktuellen Stand nach jedem Verbindungsaufbau gemäß Kapitel 9.3 herstellen.

SHOULD Innerhalb einer bestehenden Verbindung SOLLTEN Clients Duplikate und verspätete Nachrichten anhand von `messageId` und `revision` erkennen und idempotent verarbeiten.

9.2 Wiederanlauf und Backoff

MUST Clients MÜSSEN mit Verbindungsabbrüchen umgehen können und benötigte Subscriptions sowie technische Verfügbarkeitsmeldungen nach einem erneuten Verbindungsaufbau erneut einrichten.

MUST Bei fehlgeschlagenem oder abgebrochenem Verbindungsaufbau MÜSSEN Clients erneute Versuche mit exponentiellem Backoff durchführen: Startwartezeit 1 Sekunde, Verdopplung je Fehlversuch, Obergrenze 300 Sekunden und zusätzlicher Jitter von 0 bis 50 Prozent. Pro Wiederanlaufzyklus DÜRFEN höchstens 20 aufeinanderfolgende Verbindungsversuche erfolgen. Danach MUSS der Client die automatischen Versuche beenden und eine betriebliche Störung beziehungsweise einen manuellen oder extern gesteuerten Neustart auslösen. Ein erfolgreicher Verbindungsaufbau setzt den Zähler zurück.

MUST NOT Nach Close-Code 4001 DARF ein automatischer Wiederverbindungsversuch NICHT erfolgen, bevor die Ursache behoben ist. Nach 1008, 4000, 4003 oder 4004 SOLLTE der Client zunächst die Ursache prüfen und gegebenenfalls Last, Verarbeitungsgeschwindigkeit oder Konfiguration korrigieren.

MUST Enthält eine fehlgeschlagene HTTP-Antwort einen gültigen `Retry-After`-Wert, MUSS der Client mindestens bis zu diesem Zeitpunkt beziehungsweise für diese Dauer warten. Maßgeblich ist die längere Wartezeit aus `Retry-After` und dem berechneten Backoff.

SHOULD Nach einer geplanten Beendigung durch den Server (Close-Code 1001) SOLLTEN Clients die erste Wiederverbindung frühestens nach 30 Sekunden versuchen, um Lastspitzen zu vermeiden.

MUST Nach Close-Code 4005 MUSS der Client die Verbindung mit einer neuen H2-Transaction-Id und einem zufälligen, gleichverteilten Jitter von 1 bis 30 Sekunden neu aufbauen. Der exponentielle Backoff gilt erst, wenn dieser planmäßige Wiederverbindungsversuch fehlschlägt.

Die planmäßige Rotation mit Close-Code 4005 ist eine neutrale Beendigung im Sinne von Kapitel 7.4; für betroffene Verfügbarkeitsmeldungen beginnt eine neue vollständige Karenzzeit von 120 Sekunden.

9.3 Resynchronisation über die synchrone Web-API

MUST Nach jedem Verbindungsaufbau MÜSSEN Clients den aktuellen Stand der für sie relevanten Verzeichniseinträge herstellen. Hierzu ist die in der `subscribeAck` übermittelte Revision mit dem lokalen Stand zu vergleichen; bei Abweichung oder Unkenntnis des lokalen Standes MUSS der Verzeichniseintrag über die synchrone Web-API des Verzeichnisdienstes abgerufen werden.

SHOULD Clients SOLLTEN auch bei erkannten Lücken in der Revisionsfolge während einer bestehenden Verbindung die betroffenen Einträge über die synchrone Web-API resynchronisieren, so weit eine lückenlose Verarbeitung nicht sichergestellt werden kann (Teil 2, Kapitel 5.6 [API-GL]).

MUST Bei einem isolierten Wegfall einer technischen Verfügbarkeitsmeldung infolge eines Verbindungsabbruchs MUSS der Client, der den Zustand `confirmed` aufrechterhalten will, die Verfügbarkeitsmeldung innerhalb der Karenzzeit gemäß Kapitel 7.4 erneut aufbauen. Während `monitoringUnavailable` besteht, MUSS der Client den Wiederanlauf gleichwohl gemäß Kapitel 9.2 fortsetzen; der Server DARF abgelaufene Fristen nicht rückwirkend anwenden.

MUST Nach einer Wiederverbindung MUSS ein Client mit aktiven technischen Verfügbarkeitsmeldungen die erforderlichen `availabilityStart`-Nachrichten vor allen `subscribe`-Nachrichten senden. `availabilityStart`-Nachrichten behalten auch bei Verzögerungen durch das Nachrichtenratenlimit Vorrang; `subscribe`-Nachrichten DÜRFEN erst nach Übermittlung aller erforderlichen `availabilityStart`-Nachrichten gesendet werden.

10. Betriebsparameter

Die folgenden Parameter gelten verbindlich für den Betrieb der WebSocket-API. Änderungen der Parameter erfolgen ausschließlich im Rahmen des Änderungsmanagements gemäß Teil 1, Kapitel 4.2 [API-GL].

Parameter	Wert	Beschreibung
Maximale Nachrichtengröße	1.048.576 Byte (1 MiB)	Obergrenze je vollständig zusammengesetzter, unkomprimierter Nachricht einschließlich Envelope.

Parameter	Wert	Beschreibung
Maximale Größe entry	921.600 Byte (900 KiB)	Obergrenze des UTF-8-serialisierten vollständigen Verzeichniseintrags innerhalb einer notification.
Maximale Länge signature	4.096 Byte	Obergrenze der Envelope-Eigenschaft signature; größere Werte MÜSSEN mit ERR-SIG-001 abgelehnt werden.
Heartbeat-Intervall (fachlich)	60 Sekunden	Sendeintervall je aktiver availabilityHeartbeat-Meldung. Mehrere Heartbeats sind gleichmäßig zu verteilen.
Heartbeat-Timeout (fachlich)	180 Sekunden	Zeit seit dem letzten serverseitigen Empfang, nach der bei ordnungsgemäß arbeitender Überwachungsinfrastruktur ein isolierter Heartbeat-Ausfall festgestellt wird. Entspricht $3 \times$ Heartbeat-Intervall ($180 = 3 \times 60$ Sekunden).
Karenzzeit	120 Sekunden	Zusätzliche Frist bei endpunktspezifischem Wegfall bis zur automatischen Statusänderung. Gilt nicht bei monitoringUnavailable.
Mass-Loss-Beobachtungsfenster	60 Sekunden	Zeitraum zur Ermittlung korrelierter Wegfälle je Umgebung.
Mass-Loss-Mindestanzahl	20	Mindestens 20 korrelierte Wegfälle innerhalb des Beobachtungsfensters.
Mass-Loss-Mindestanteil	10 %	Zusätzlich müssen mindestens 10 Prozent der zu Beginn des Beobachtungsfensters aktiven Verfügbarkeitsmeldungen aufgrund derselben oder einer technisch korrelierten Fehlerursache betroffen sein.
Circuit-Breaker-Stabilitätszeit	300 Sekunden	Minstdauer eines stabilen Zustands operational vor Rücksetzung.
connectionId-Nachnutzungssperre	600 Sekunden nach Verbindungsende	Eine verwendete connectionId darf für dieselbe MP-ID und Umgebung

Parameter	Wert	Beschreibung
		während dieser Zeit nicht erneut gebunden werden.
Maximale Verbindungsdauer	24 Stunden	Obergrenze der Dauer einer WebSocket-Verbindung. Der Server beendet sie anschließend planmäßig mit Close-Code 4005. Nach der Rotation gilt ein zufälliger Wiederverbindungs-Jitter von 1 bis 30 Sekunden (Kapitel 9.2).
Ping-Intervall (technisch)	30 Sekunden	Intervall der serverseitigen WebSocket-Ping-Frames; Clients MÜSSEN mit Pong antworten.
Idle-Timeout	300 Sekunden	Zeit ohne empfangene Anwendungsnachricht oder Pong, nach der der Server mit Close-Code 4002 beendet.
Antwort-Timeout	30 Sekunden	Zeit, innerhalb derer der Server eine korrelierbare Client-Nachricht mit Ack oder error beantworten MUSS.
Maximale Benachrichtigungsverzögerung	30 Sekunden	Maximale Zeit zwischen abgeschlossener Änderung im Verzeichnisdienst und Übergabe der notification an eine schreibbereite Verbindung im Normalbetrieb.
Maximale Subscriptions je Verbindung	1.000	Obergrenze gleichzeitig aktiver Subscriptions.
Maximale Verfügbarkeitsmeldungen je Verbindung	200	Obergrenze gleichzeitig aktiver technischer Verfügbarkeitsmeldungen.
Maximale Verbindungen je Marktpartner	10	Obergrenze gleichzeitiger WebSocket-Verbindungen je MP-ID und Umgebung, unabhängig von der Marktrolle.
Maximale Verbindungsaufbauversuche	20 je 60 Sekunden	Obergrenze der Upgrade-Versuche je MP-ID und Umgebung; Überschreitung führt zu HTTP 429 (serverseitiges Upgrade-Rate-Limit, unabhängig von der clientseitigen Backoff-Abbruchgrenze).

Parameter	Wert	Beschreibung
Maximale Nachrichtenrate	50 Nachrichten je 10 Sekunden	Obergrenze der Client-Anwendungsnachrichten je Verbindung im gleitenden Fenster. Ping-, Pong- und Close-Frames zählen nicht.
Backoff (Wiederanlauf)	1 s Start, Faktor 2, max. 300 s, Jitter 0–50 %	Wartezeit zwischen Wiederverbindungsversuchen.
Maximale Wiederverbindungsversuche	20 je Wiederanlaufzyklus	Clientseitige Backoff-Abbruchgrenze: Nach 20 aufeinanderfolgenden Fehlversuchen sind automatische Versuche zu beenden und betrieblich zu eskalieren. Unabhängig vom serverseitigen Upgrade-Rate-Limit.

MUST Der Server MUSS mindestens Verbindungsaufbau und -ablehnung, H2-Transaction-Id beziehungsweise `connectionId`, authentifizierte MP-ID und Marktrolle, `messageId` und `correlationId` der verarbeiteten Anwendungsnachrichten, Änderungen an Subscriptions, Verfügbarkeitsmeldungen und `availabilityMonitoringStatus`, Rate-Limit-Ereignisse, Close-Codes, Circuit-Breaker-Ereignisse sowie automatische Statusänderungen nachvollziehbar protokollieren. Schutzbedürftige Inhalte DÜRFEN nur im erforderlichen Umfang erfasst werden.

SHOULD Clients SOLLTEN `messageId`, `correlationId`, Close-Codes, Wiederverbindungsversuche und Resynchronisationen so protokollieren, dass betriebliche Störungen ohne Speicherung unnötiger fachlicher Nutzdaten analysiert werden können.

11. JSON-Schemata

MUST Nachrichten MÜSSEN gegen die nachfolgenden Client- beziehungsweise Server-Schemata nach JSON Schema Draft 2020-12 validierbar sein. Die normative maschinenlesbare Fassung wird über das Repository der API-Governance bereitgestellt [H2-AG-API]; bei Abweichungen geht die dort veröffentlichte Fassung vor.

11.1 Validierungsgrundsätze

MUST Nachrichten des Clients an den Server MÜSSEN auf Envelope- und Payload-Ebene strikt gegen das Client-Schema validiert werden. Nicht spezifizierte Eigenschaften sind unzulässig und MÜSSEN mit `ERR-VAL-001` abgelehnt werden. Abweichungen, die ausschließlich die Eigenschaft `signature` betreffen (Fehlen, falscher Datentyp, Pattern- oder Längenverletzung), MÜSSEN abweichend davon als `ERR-SIG-001` behandelt werden. Vor der vollständigen Typvalidierung MUSS der Server prüfen, ob `type` als `string` vorhanden und in Client-Richtung bekannt ist; ein unbekannter Typ führt zu `ERR-MSG-001`.

MUST Das Server-Schema beschreibt für bekannte Server-Nachrichtentypen die verbindliche Mindeststruktur. Zusätzliche optionale Eigenschaften sind zulässig und MÜSSEN ignoriert werden. Unbekannte Server-Nachrichtentypen MÜSSEN nach erfolgreicher Envelope-Prüfung ignoriert werden. Bei einer unbekannten `eventType` innerhalb einer `notification` gilt die Resynchronisationsregel aus Kapitel 6.4. Eine bekannte Server-Nachricht, die ihre Mindeststruktur verletzt, DARF der Client NICHT verarbeiten; er SOLLTE die Verbindung mit Close-Code 1008 beenden und eine betriebliche Störung melden.

MUST NOT Die Eigenschaft `signature` DARF in keiner Server-Nachricht enthalten sein und MUSS im Server-Schema unabhängig von der Erweiterbarkeit sonstiger Eigenschaften durch ein boolesches Falsch-Schema formal verboten werden.

MUST Die Eigenschaft `notification.payload.entry` MUSS über einen verbindlichen `$ref` auf das vollständige `DirectoryRecord`-Schema der synchronen Web-API validiert werden. Des- sen kanonische `$id` MUSS `urn:h2:directory:record:v1` lauten und das Schema MUSS zusammen mit den WebSocket-Schemata im selben Governance-Artefaktbündel bereitgestellt werden.

MUST NOT Implementierungen DÜRFEN das `DirectoryRecord`-Schema NICHT zur Laufzeit aus einer unkontrollierten Netzquelle nachladen. Die anhand der `$id` aufgelöste, freigegebene Fassung MUSS lokal beziehungsweise aus einem kontrollierten Schema-Registry-Artefakt verwendet werden.

MUST Implementierungen MÜSSEN die JSON-Schema-Formatprüfung als verbindliche Assertion aktivieren. Insbesondere MÜSSEN `format = uuid` und `format = date-time` tatsächlich validiert werden.

11.2 Kennungsformate

Eigenschaft	Format	Beschreibung
messageId / correlationId	UUID Version 4, lowercase	Pattern: <code>^[0-9a-f]{8}-[0-9a-f]{4}-4[0-9a-f]{3}-[89ab][0-9a-f]{3}-[0-9a-f]{12}\$</code> . correlationId darf zusätzlich null sein. UUID Version 4 ist bewusst gewählt; ein Zeitbezug wie bei UUID Version 7 ist für messageId weder erforderlich noch erwünscht.
timestamp / validUntil	RFC 3339 date-time in UTC	MUSS mit dem Suffix Z übertragen und als date-time validiert werden.
providerId	string, <code>^[0-9]{13}\$</code>	13-stellige numerische MP-ID.
apiId	string, <code>^H2API[0-9]{5}\$</code>	API-Kennung gemäß API-Katalog der API-Governance.
majorVersion	integer, 1 bis 2147483647	Positive Hauptversion des API-Webdienstes.
revision	integer, 1 bis 9007199254740991	I-JSON-interoperable, streng monoton steigende Revision je Verzeichnisschlüssel.
notification.payload.reason	lowerCamelCase string	Maschinenlesbarer Code mit Pattern <code>^[a-z][a-zA-Z0-9]{2,63}\$</code> . Zulässige Werte gemäß Kapitel 6.2; unbekannte, musterkonforme Werte in Server-Nachrichten MÜSSEN toleriert werden.
availabilityStop.payload.reason	string (enum)	Zulässige Werte: <code>serviceUnavailable</code> , <code>clientShutdown</code> , <code>connectionMigration</code> , <code>plannedRestart</code> , <code>other</code> . Unbekannte Werte in Client-Nachrichten MÜSSEN mit <code>ERR-VAL-001</code> abgelehnt werden.

11.3 Client-Nachrichtenschema

Das kanonische Client-Nachrichtenschema wird als Datei `ws-client-message.schema.yaml` mit der Schema-ID `urn:h2:directory:ws:client-message:v1` im gemeinsamen Governance-Artefaktbündel veröffentlicht.

MUST Das Schema MUSS ausschließlich subscribe, unsubscribe, availabilityStart, availabilityHeartbeat und availabilityStop zulassen. Client-Nachrichten MÜSSEN auf Envelope- und Payload-Ebene geschlossen sein. correlationId MUSS null sein; signature MUSS vorhanden sein. availabilityStop.payload.reason MUSS einen der in Kapitel 7.3 festgelegten Werte enthalten.

MUST Implementierungen MÜSSEN die zu dieser Schnittstellenversion veröffentlichte Fassung verwenden und die Schema-ID lokal oder über eine kontrollierte Schema-Registry auflösen. Ein Laufzeitabruf aus einer unkontrollierten Netzquelle ist unzulässig. Client-Schema ist geschlossen und umfasst ausschließlich die in Kapitel 4.3 definierten Client-Nachrichtentypen. Für alle Client-Nachrichten MUSS correlationId = null gelten. Jede signaturpflichtige Client-Nachricht MUSS die Eigenschaft signature gemäß Kapitel 3.5 enthalten. availabilityStop verwendet einen eigenen Payload mit verpflichtendem reason.

11.4 Server-Nachrichtenschema

Das kanonische Server-Nachrichtenschema wird als Datei ws-server-message.schema.yaml mit der Schema-ID urn:h2:directory:ws:server-message:v1 im gemeinsamen Governance-Artefaktbündel veröffentlicht.

MUST Das Schema MUSS für subscribeAck, unsubscribeAck, availabilityAck, notification und error die verbindliche Mindeststruktur festlegen. Zusätzliche optionale Servereigenschaften und unbekannte künftige Server-Nachrichtentypen MÜSSEN gemäß Kapitel 2.3 und 11.1 toleriert werden. Die Eigenschaft signature MUSS in sämtlichen Server-Nachrichten formal verboten sein.

MUST notification.payload.entry MUSS auf urn:h2:directory:record:v1#/\$defs/DirectoryRecord verweisen. directory-record.schema.yaml MUSS im selben Governance-Artefaktbündel veröffentlicht und kontrolliert aufgelöst werden.

11.5 Verarbeitungsreihenfolge bei Client-Nachrichten

MUST Der Server MUSS eine Client-Nachricht in folgender Reihenfolge prüfen und verarbeiten:

1. Framegröße, UTF-8-Kodierung und JSON-Syntax prüfen.
2. Nachrichtenrate prüfen.
3. Vorhandensein und Richtung von type sowie die Schema-Konformität einschließlich Format der messageId prüfen. Abweichungen, die ausschließlich die Eigenschaft signature betreffen, sind für Schritt 4 zurückzustellen.

4. Vorhandensein, Datentyp, Länge und syntaktische Struktur der Eigenschaft `signature` prüfen; anschließend die Signaturrepräsentation einschließlich der gebundenen `connectionId` bilden und JWS, Zertifikat sowie Signaturzeitfenster gemäß Kapitel 3.5 und [JWS-SPEC] prüfen.
5. `messageId` für die aktuelle `connectionId` atomar reservieren; eine bereits reservierte `messageId` mit `ERR-MSG-002` ablehnen.
6. Berechtigung, fachlich-semantische Voraussetzungen und Idempotenzlage prüfen. Limits für aktive Subscriptions oder Verfügbarkeitsmeldungen DÜRFEN nur angewendet werden, wenn ein neuer Zustand angelegt wird; idempotente Wiederholungen DÜRFEN wegen ausgeschöpfter Limits NICHT abgelehnt werden.
7. Operation idempotent ausführen und mit `Ack` oder `error` beantworten.

MUST Ein unbekannter oder richtungswidriger `type` führt zu `ERR-MSG-001`, eine doppelte `messageId` zu `ERR-MSG-002`, eine sonstige Schemaverletzung zu `ERR-VAL-001` und ein Fehler von `signature` oder `connectionId` zu `ERR-SIG-001`. Zustandsänderungen DÜRFEN erst nach erfolgreichem Abschluss aller Prüfungen erfolgen; Fehler DÜRFEN keinen Teilzustand hinterlassen.

12. Beispielablauf

Der folgende Ablauf zeigt exemplarisch die Nutzung der WebSocket-API durch einen Client, der einen fremden Verzeichniseintrag abonniert und für einen eigenen Verzeichniseintrag die technische Verfügbarkeit meldet:

1. Der Client baut die mTLS-gesicherte WSS-Verbindung zum Endpunkt `wss://<Host>/v1/ws/subscriptions` als HTTP/1.1-GET-Upgrade auf. Der Request enthält `H2-Business-Process: directoryWebSocket`
2. Der Server prüft TLS- und SIG-Zertifikat, MP-ID, Marktrolle und Berechtigung, bestätigt das Subprotokoll `h2.directory.v1` und `H2-API-Version: 1.0.0`, schließt das Upgrade ab und bindet `H2-Transaction-Id` als `connectionId` sowie das SIG-Zertifikat als Signaturschlüssel der Verbindung.
3. Der Client sendet `subscribe` (signiert gemäß Kapitel 3.5) für den Verzeichniseintrag (`providerId`, `apiId`, `majorVersion`) und erhält `subscribeAck` mit der aktuellen Revision 17.
4. Der Client vergleicht die Revision mit seinem lokalen Stand; bei Abweichung ruft er den Verzeichniseintrag über die synchrone Web-API ab.
5. Der Client sendet `availabilityStart` für seinen eigenen Verzeichniseintrag und anschließend alle 60 Sekunden `availabilityHeartbeat` (jeweils signiert gemäß Kapitel 3.5); der Server bestätigt jeweils mit `availabilityAck` und `validUntil`.
6. Der Betriebsstatus des abonnierten Eintrags ändert sich; der Server sendet `notification` mit `eventType = statusChanged` und Revision 18.
7. Die Verbindung bricht ab. Bei einem isolierten Ausfall beginnt die Karenzzeit; bei einem korrelierten oder serverseitigen Infrastrukturausfall wird `monitoringUnavailable` gesetzt und der Mass-Loss-Circuit-Breaker verhindert automatische down-Übergänge. Der Client verbindet mit begrenztem Backoff neu und richtet die Verfügbarkeitsmeldungen erneut ein; dabei sendet er nach der Wiederverbindung gemäß Kapitel 9.3 alle `availabilityStart`-Nachrichten vor allen `subscribe`-Nachrichten.
8. Zum Betriebsende sendet der Client `availabilityStop` mit `reason = clientShutdown` und `unsubscribe` und schließt die Verbindung mit Close-Code 1000.

13. Quellen

[API-GL] API-Guideline Wasserstoffmarkt, Version 1.0a, Working Document, in der jeweils gültigen Fassung; insbesondere Teil 1, Kapitel 3.4 sowie 4.2 bis 4.5 und Teil 2, Kapitel 2, 4 und 5.

[RFC2119] Bradner, S. Key words for use in RFCs to Indicate Requirement Levels. IETF RFC 2119, March 1997. <https://www.rfc-editor.org/info/rfc2119>

[RFC8174] Leiba, B. Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words. IETF RFC 8174, May 2017. <https://www.rfc-editor.org/info/rfc8174>

[RFC6455] Fette, I. and Melnikov, A. The WebSocket Protocol. IETF RFC 6455, December 2011. <https://www.rfc-editor.org/info/rfc6455>

[RFC6066] Eastlake, D. Transport Layer Security (TLS) Extensions: Extension Definitions. IETF RFC 6066, January 2011. <https://www.rfc-editor.org/info/rfc6066>

[RFC7493] Bray, T. The I-JSON Message Format. IETF RFC 7493, March 2015. <https://www.rfc-editor.org/info/rfc7493>

[RFC3339] Klyne, G. and Newman, C. Date and Time on the Internet: Timestamps. IETF RFC 3339, July 2002. <https://www.rfc-editor.org/info/rfc3339>

[RFC9562] Davis, K., Peabody, B., and Leach, P. Universally Unique IDentifiers (UUIDs). IETF RFC 9562, May 2024. <https://www.rfc-editor.org/info/rfc9562>

[JSON-SCHEMA-2020-12] JSON Schema Organization. JSON Schema Draft 2020-12, Core and Validation Specifications. <https://json-schema.org/draft/2020-12>

[JWS-SPEC] Technische Spezifikation zur JWS-Inhaltsdatensicherung für API-Webdienste im Wasserstoffmarkt, in der jeweils gültigen Fassung; für diese Schnittstelle maßgeblich für die Signatur von Nachrichten innerhalb der WebSocket-Verbindung gemäß Kapitel 3.5.

[TR03116-3] Bundesamt für Sicherheit in der Informationstechnik. BSI TR-03116-3 – Kryptographische Vorgaben für Projekte der Bundesregierung, Teil 3: Intelligente Messsysteme, in der jeweils gültigen Fassung.

[CP-SM-PKI] Bundesamt für Sicherheit in der Informationstechnik. Certificate Policy der Smart Metering PKI. Version 1.1.2, 25.01.2023.

[RzÜ-API] BDEW. Regelungen zum Übertragungsweg für API-Webdienste. Version 1.2, Publikationsdatum 01.04.2025, anzuwenden ab 01.10.2025.

[RzV] BDEW. Regelungen zum Verzeichnisdienst. Version 1.1, Publikationsdatum 01.04.2025, anzuwenden ab 01.10.2025.

[H2-AG-API] GitHub-Organisation H2-AG-API für die Anlage der API-Kennungen und die Bereitstellung maschinenlesbarer Schemata: <https://github.com/H2-AG-API>

Version	Datum	Änderung	Autor
0.9	28.09.2026	Initiale Erstellung	Arbeitsgruppe API